



RISC-V Project

Raghavendran Ganesan

21BEC0996

ACKNOWLEDGEMENT:

I, **Raghavendran Ganesan {21BEC0996}**, would like to acknowledge that the RISC-V RV321 architecture project presented herein was solely undertaken and completed by me. This project provided me with the opportunity to delve deeply into the intricacies of RISC-V architecture and semiconductor design, enabling me to learn and grow significantly throughout this endeavor.

I am immensely thankful to **Mr. Raghavendra H** for his mentorship, patience, and insightful feedback, which significantly enhanced the quality of my project. His dedication to teaching and willingness to share knowledge have been instrumental in my learning journey.

Lastly, I would like to express my gratitude to **Maven Silicon** for providing a conducive environment for learning and for their commitment to fostering excellence in the field of semiconductor design. The resources and opportunities offered by Maven Silicon have been instrumental in expanding my understanding of RISC-V architecture and its applications.

INDEX:

S.No	CONTENTS	PAGE NO
1	ABSTRACT	4
2	RISC-V TOP MODULE	5
3	PC MUX	7
4	REGISTER BLOCK 1	9
5	IMMEDIATE GENERATOR	11
6	IMMEDIATE ADDER	13
7	INTEGER FILE	15
8	WRITE ENABLE GENERATOR	17
9	INSTRUCTION MUX	19
10	BRANCH UNIT	21
11	DECODER	23
12	MACHINE CONTROL	25
13	CSR FILE	27
14	REGISTER BLOCK 2	29
15	STORE UNIT	31
16	LOAD UNIT	33
17	ALU	35
18	WB MUX SEL UNIT	37
19	CONCLUSION	39

ABSTRACT:

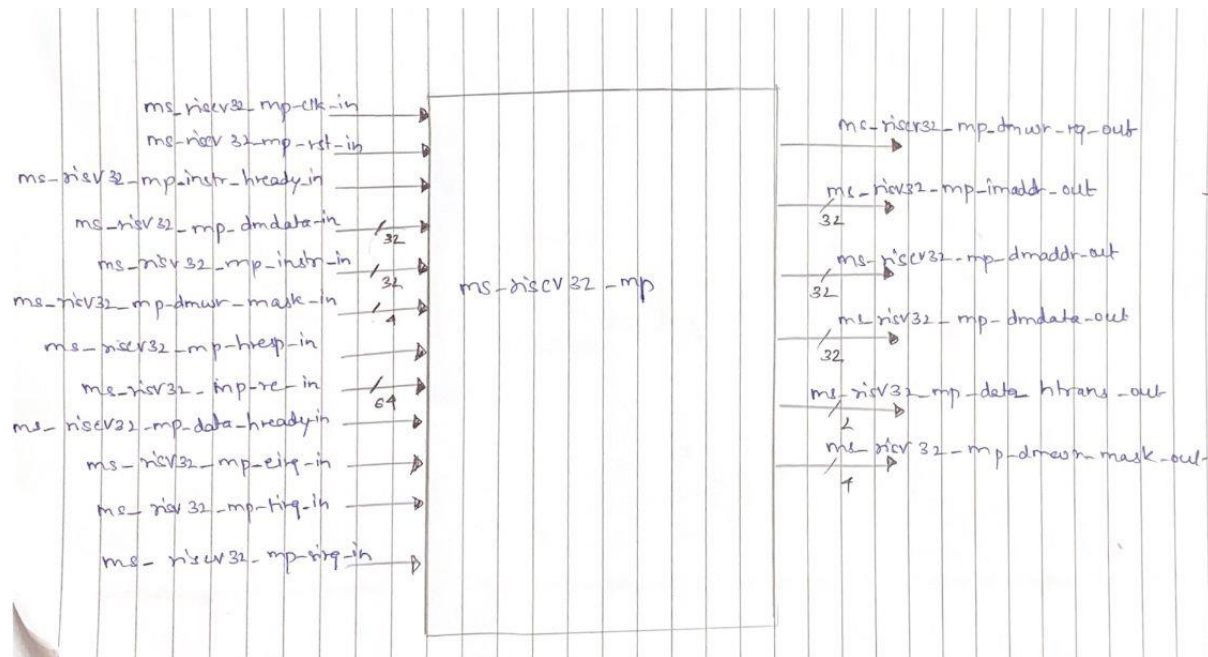
RISC-V is a new instruction set architecture (ISA) that was originally designed to support computer architecture research and education. as a streamlined microprocessor design found in electronic devices. Its simplified approach speeds up processing and reduces energy consumption. Unlike closed designs, RISC-V RV32I is open-source, encouraging collaboration and innovation. This openness allows people to understand, modify, and create new technologies, advancing computing.

I applied my knowledge to construct a RISC-V RV32I architecture, employing software such as Model Sim Altera, Quartus Prime, Sophus Connect, and MobaXTerm for the project's development and analysis.

The project contains the top module of RISC-V RV32I Architecture along with the submodules needed for its completion.

RISC-V TOP MODULE:

BLOCK DIAGRAM:



FUNCTIONALITY:

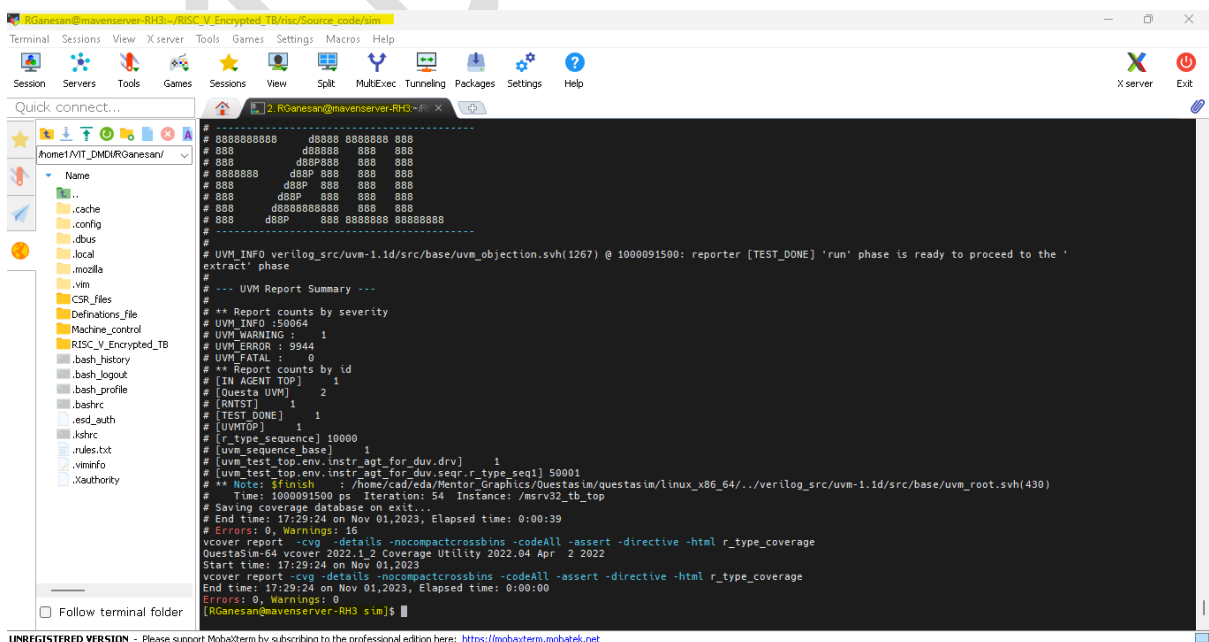
- To enable the implementation and testing of all RV32I instructions, all submodules are built here with the input.
- The top module receives a variety of inputs, which result in important outputs, which are subsequently used as inputs by several sub-blocks.
- The top module supervises the operation of all sub-modules and coordinates.
- all tasks like retrieve, decode, execute, memory access, and write back.

FILES PRESENT IN MobaXTerm Server:

```
[RGanesan@mavenserver-RH3 Source_code]$ cd rtl
[RGanesan@mavenserver-RH3 rtl]$ ls
csr_data_mux_unit.v      mie_reg.v              msrv32_decoder.v       msrv32_machine_control.v  mstatus_reg.v
data_wr_mux_unit.v      mip_reg.v              msrv32_immediate_adder.v  msrv32_pc.v              mtval_reg.v
machine_counter_setup.v  misa_and_pre_data.v    msrv32_imm_generator.v  msrv32_reg_block1.v      mtvec_reg.v
machine_counter.v        msrv32_alu.v           msrv32_instruction_mux.v  msrv32_reg_block2.v      wb_mux_sel_unit.v
mcause_reg.v            msrv32_branch_unit.v  msrv32_integer_file.v    msrv32_store_unit.v      wr_en_generator.v
mepc_and_mscratch_reg.v  msrv32_csr_file.v     msrv32_load_unit.v       msrv32_top_wrapper.v
```

```
2. RGanesan@mavenserver-RH3: ~/R/ X
Top level modules:
msrv32_alu
msrv32_branch_unit
msrv32_csr_file
msrv32_decoder
msrv32_immediate_adder
msrv32_imm_generator
msrv32_instruction_mux
msrv32_integer_file
msrv32_load_unit
msrv32_machine_control
msrv32_pc
msrv32_reg_block1
msrv32_reg_block2
msrv32_store_unit
msrv32_wb_mux_sel_unit
msrv32_wr_en_generator
msrv32_tb_top
```

OUTPUT WAVEFORM:

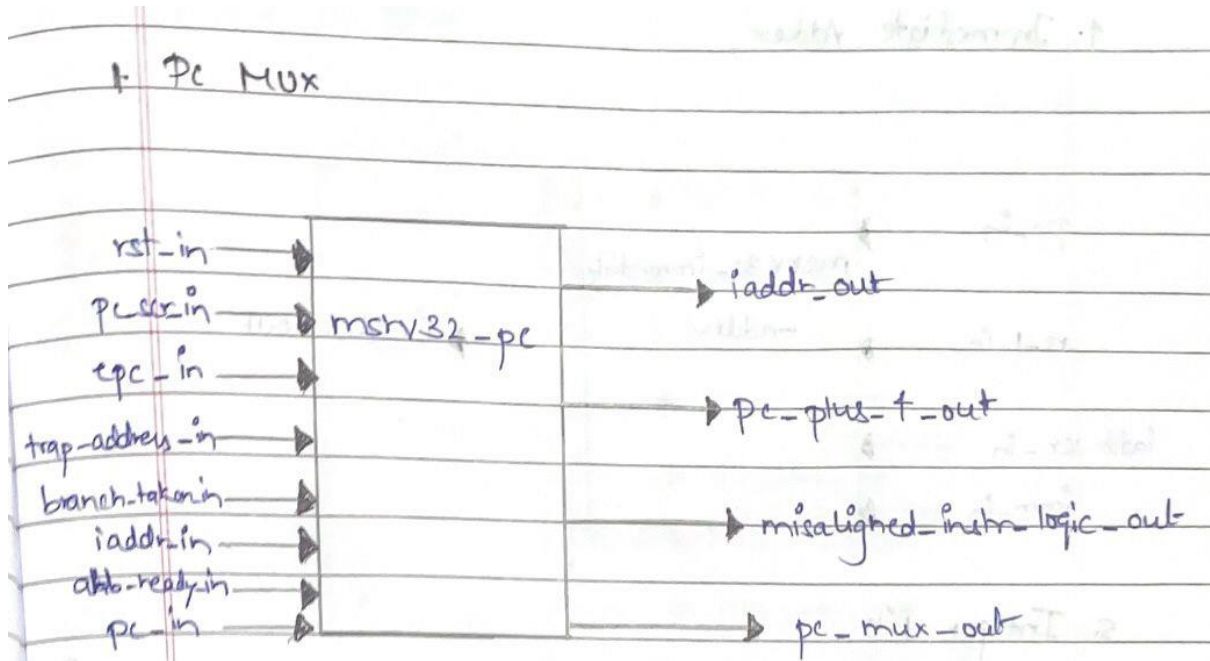


```
RGanesan@mavenserver-RH3:~/RISC_V_Encrypted_TB/VeriSource_code/um
Terminal Sessions View Xserver Tools Games Settings Macros Help
Session Servers Tools Games Sessions View Split MultExec Tunneling Packages Settings Help
Quick connect...
# 888888888 d8888 8888888 888
# 888 d88888 888 888
# 888 d88P888 888 888
# 8888888 d88P 888 888 888
# 888 d88P 888 888 888
# 888 d88P 888 888 888
# 888 d8888888888 888 888
# 888 d88P 888 8888888 8888888
#
# UVM INFO veril_src/uvvm-1.1d/src/base/uvvm_objectcion.svh(1267) @ 1000091500: reporter [TEST_DONE] 'run' phase is ready to proceed to the '
extract' phase
#
# --- UVM Report Summary ---
#
# ** Report counts by severity
# UVM INFO : 50064
# UVM WARNING : 1
# UVM ERROR : 9944
# UVM FATAL : 0
# ** Report counts by id
# [IN AGENT TOP] 1
# [Questasim UVM] 2
# [RNTST] 1
# [TEST_DONE] 1
# [UVMTOP] 1
# [r_type_sequence] 10000
# [uvvm_sequence_base] 1
# [uvvm_test_top.env.instr_agt_for_duv.drv] 1
# [uvvm_test_top.env.instr_agt_for_duv.ssr.r_type_seq] 50001
# ** Note: Finish : /home/cad/cda/Mentor_Graphics/Questasim/questasim/linux_x86_64/.../verilog_src/uvvm-1.1d/src/base/uvvm_root.svh(430)
# Time: 1000091500 ps Iteration: 54 Instance: /msrv32_tb_top
# Saving coverage database on exit...
# End time: 17:29:24 on Nov 01,2023, Elapsed time: 0:00:39
# Errors: 0, Warnings: 0
vcover report -cvg -details -nocompactcrossbins -codeAll -assert -directive -html r_type_coverage
Questasim-64 vcover 2022.1.2 Coverage Utility 2022.04 Apr 2 2022
Start time: 17:29:24 on Nov 01,2023
vcover report -cvg -details -nocompactcrossbins -codeAll -assert -directive -html r_type_coverage
End time: 17:29:24 on Nov 01,2023, Elapsed time: 0:00:00
Errors: 0, Warnings: 0
[RGanesan@mavenserver-RH3 sim]$
```

UNREGISTERED VERSION - Please support MobaXterm by subscribing to the professional edition here: <https://mobaxterm.mobatek.net>

1. PC MUX

BLOCK DIAGRAM:



FUNCTIONALITY:

- A block in the RISC-V architecture called the PC MUX (program counter multiplexer) determines the next program counter (PC) value based on the processor's current state.
- The address of the subsequent instruction to be performed is stored in a register called the PC.
- Based on the results of the branch instruction, PC Mux chooses the address for the following instruction. If true, PC value changes correspondingly; if false, PC equals PC + 4. Even when there are branch instructions, it makes sure that the instructions are carried out in the correct order.
- PC source in, which originates from the machine control block, determines PC out value.

RTL Viewer - C:\intelFPGA_lite\17.1\msnv32_pc - msnv32_pc

File Edit View Tools Window Help

Search altera.com

Page: 1 of 1

Netlist Navigator

msnv32_pc:1

rst_in

Mux6

SEL[1:0]

DATA[3:0]

OUT

Mux7

SEL[1:0]

DATA[3:0]

OUT

Mux8

SEL[1:0]

DATA[3:0]

OUT

Mux9

SEL[1:0]

DATA[3:0]

OUT

Mux21

SEL[1:0]

DATA[3:0]

OUT

Mux30

SEL[1:0]

DATA[3:0]

OUT

Mux31

SEL[1:0]

DATA[3:0]

OUT

laddr_out[20] latch

DATA[31:20]

LATCH_ENABLE

OUT[0]

ACLK

LATCH

laddr_out[21] latch

DATA[31:21]

LATCH_ENABLE

OUT[0]

ACLK

LATCH

laddr_out[22] latch

DATA[31:22]

LATCH_ENABLE

OUT[0]

ACLK

LATCH

laddr_out[23] latch

DATA[31:23]

LATCH_ENABLE

OUT[0]

ACLK

LATCH

laddr_out[24] latch

DATA[31:24]

LATCH_ENABLE

OUT[0]

ACLK

LATCH

laddr_out[25] latch

DATA[31:25]

LATCH_ENABLE

OUT[0]

ACLK

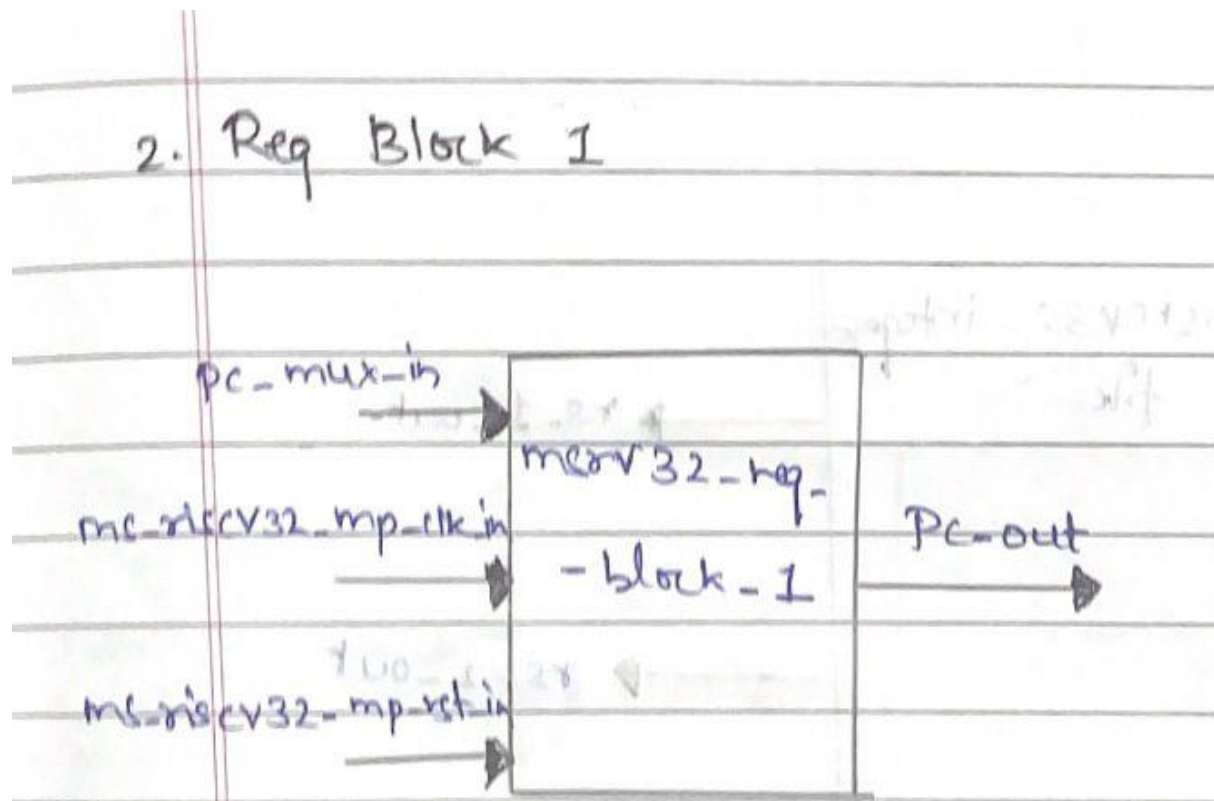
100%

00:00:00



2. REG BLOCK 1

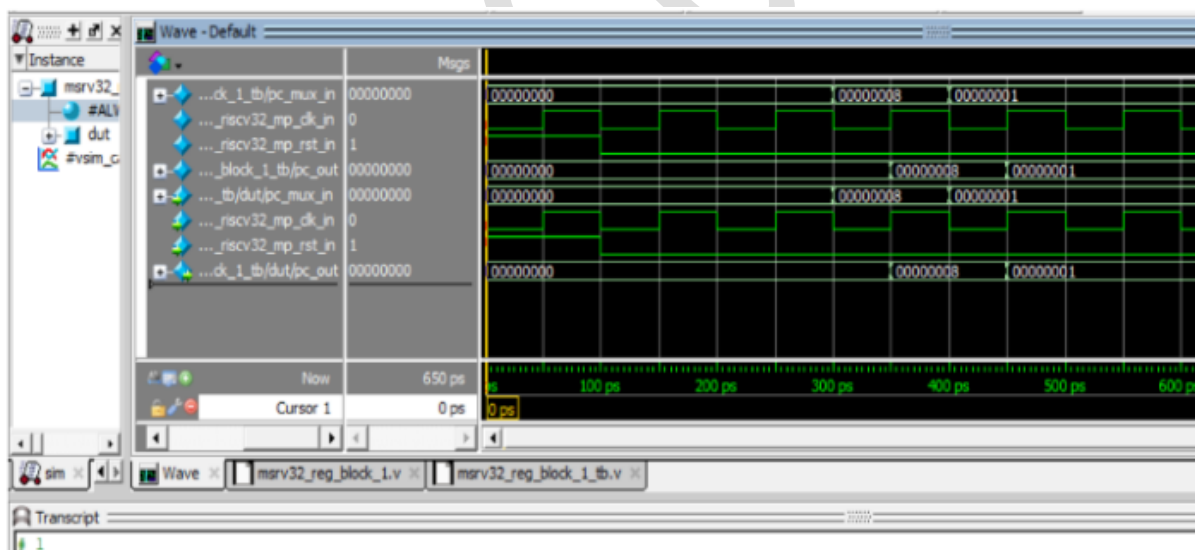
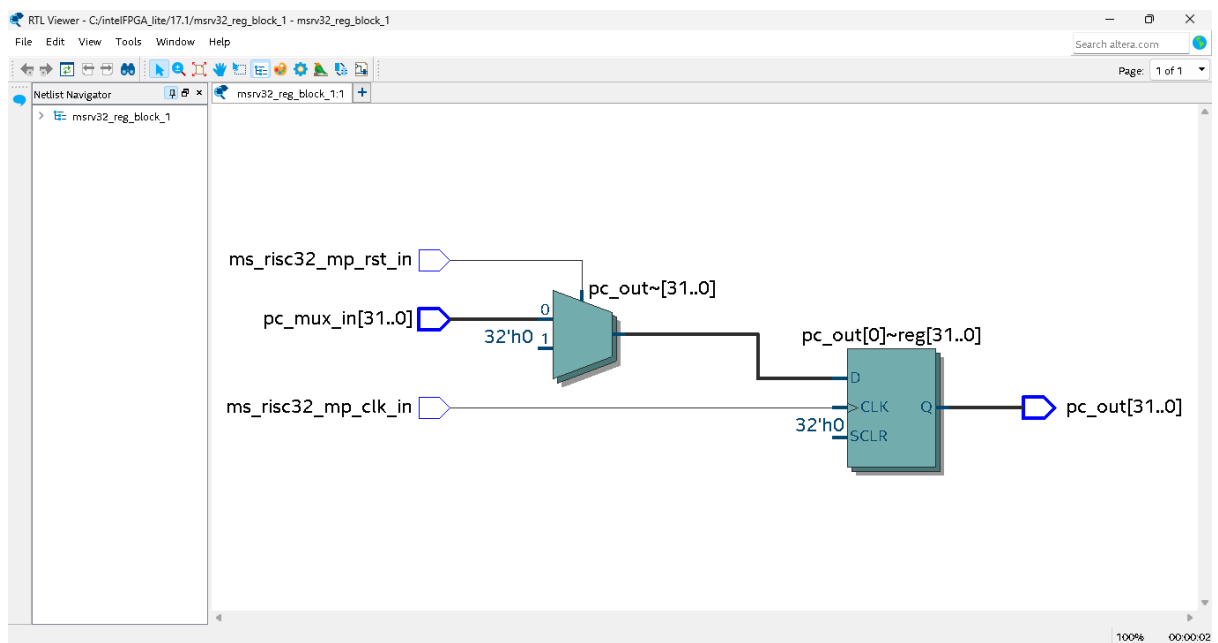
BLOCK DIAGRAM:



FUNCTIONALITY:

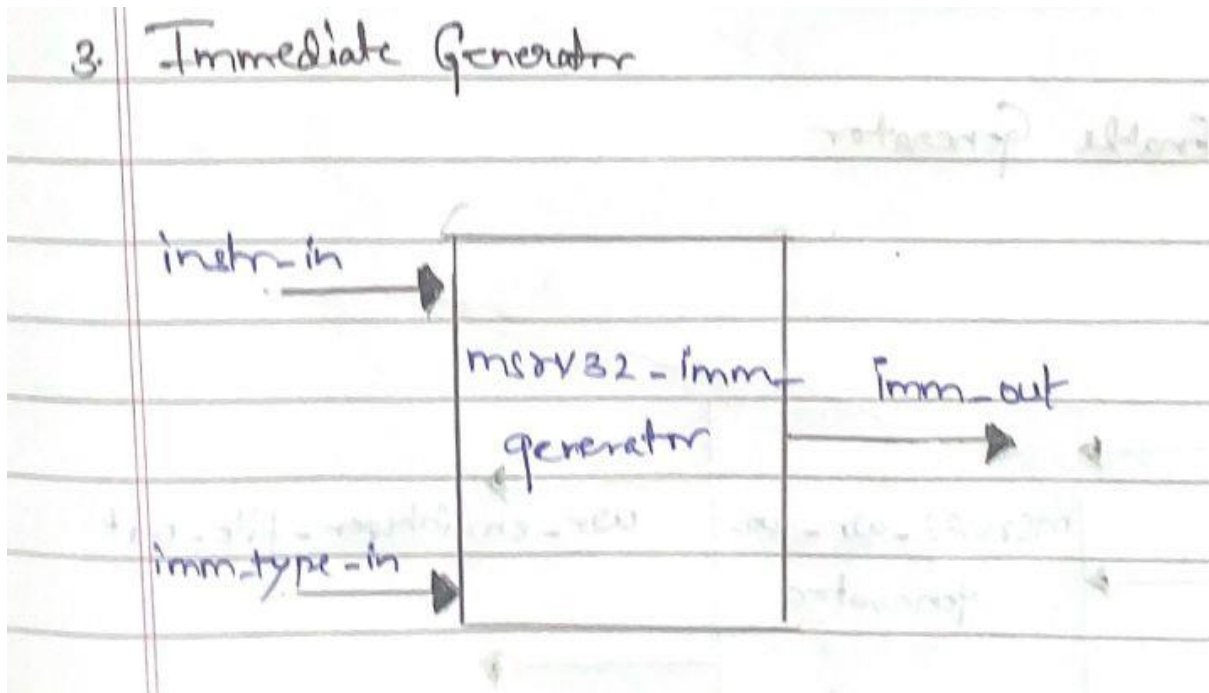
- A 32-bit value can be stored in each of the 32 registers in REG BLOCK 1. The numbers on the registers range from 0 to 31.
- The zero register, which is located at location 0, is named such because its value is always 0. Any kind of data, including integers, floating-point numbers, and memory addresses, can be stored in the remaining 31 general-purpose registers.
- It takes the output of pc mux as input and gives the pc out at the positive edge of the clock whenever reset is 0.
 - If reset is active then the pc out becomes 0

OUTPUT WAVEFORM:



3. IMMEDIATE GENERATOR

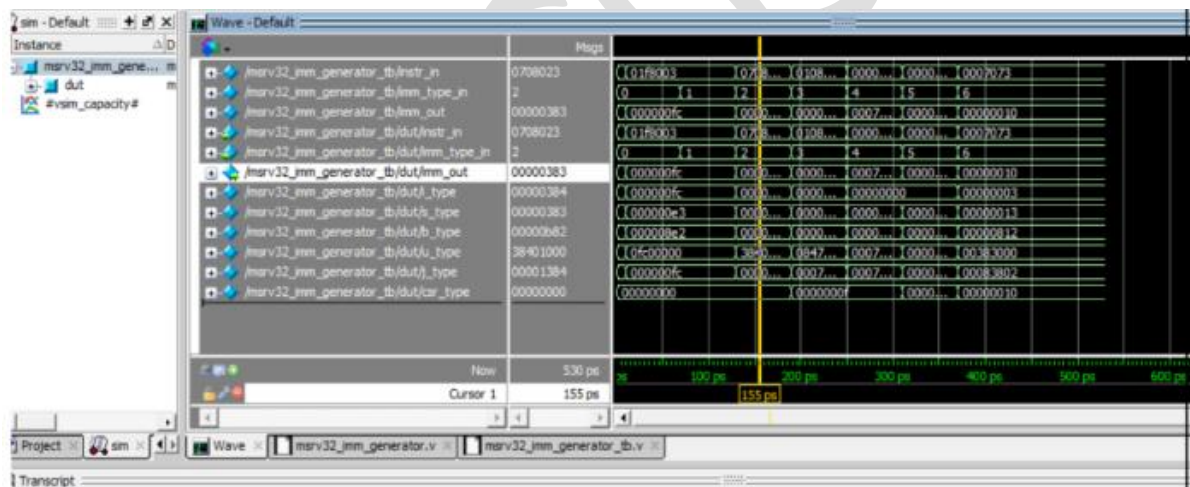
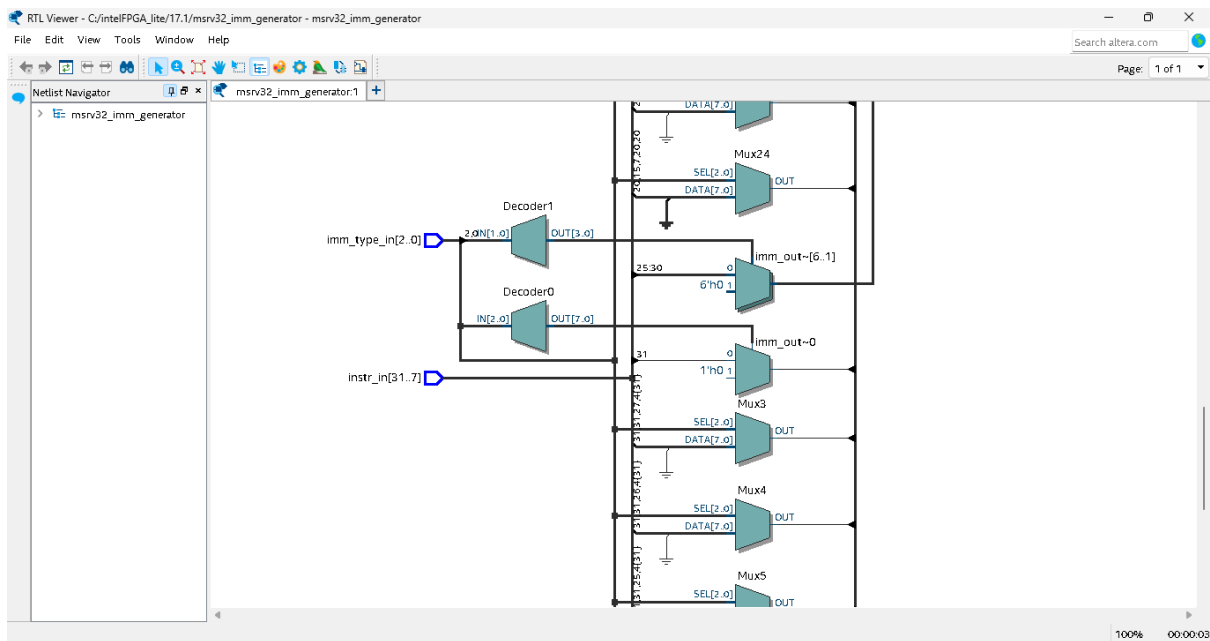
BLOCK DIAGRAM:



FUNCTIONALITY:

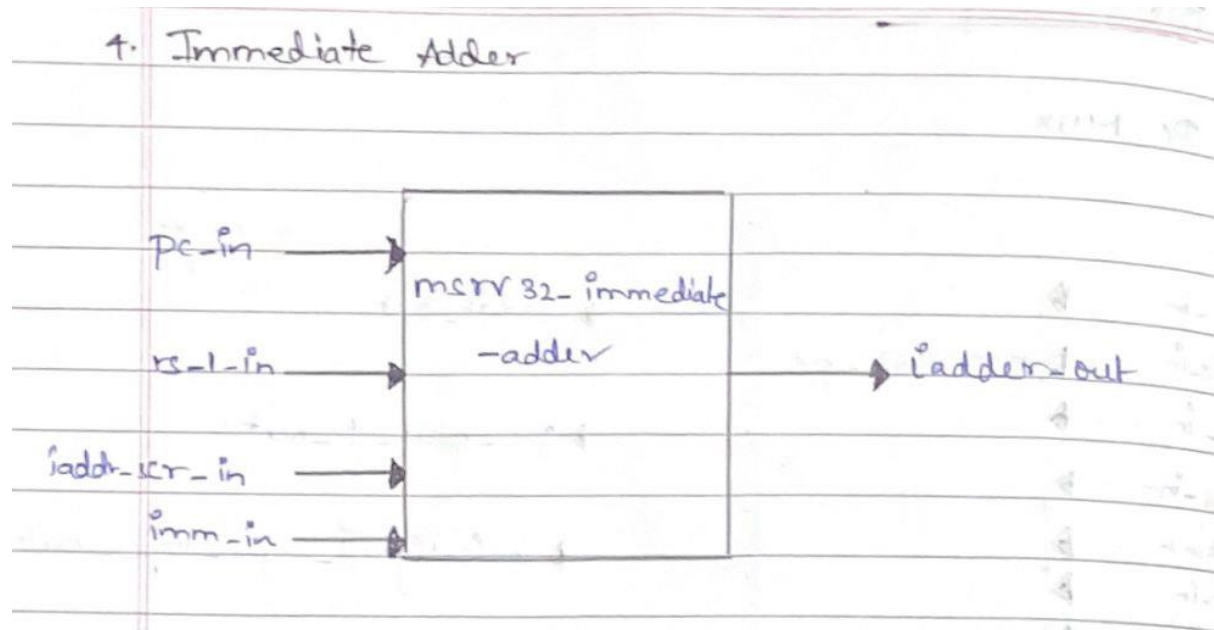
- In the RISC-V architecture, an instant generator is a circuit that generates a 32-bit immediate value from an instruction word.
- Immediate values are utilized in a variety of RISC-V instructions, including arithmetic, load and store, and branch and jump instructions.
- It generates the immediate value based on the instruction type.
- It even does necessary sign extension based on the type of instruction.

OUTPUT WAVEFORM:



4. IMMEDIATE ADDER

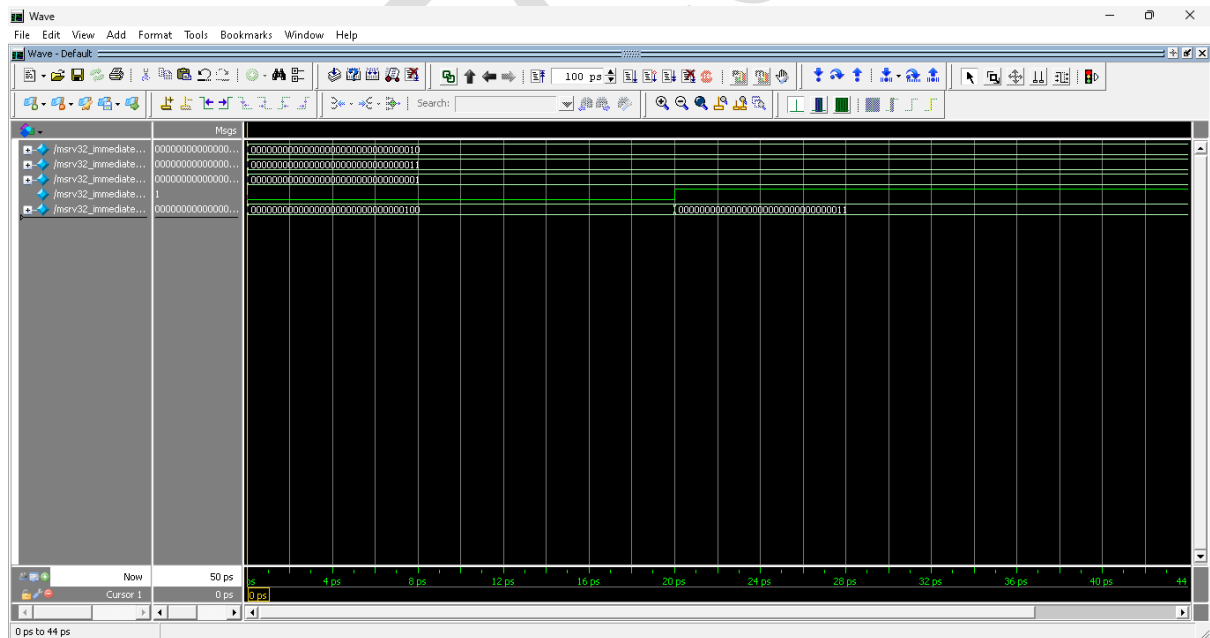
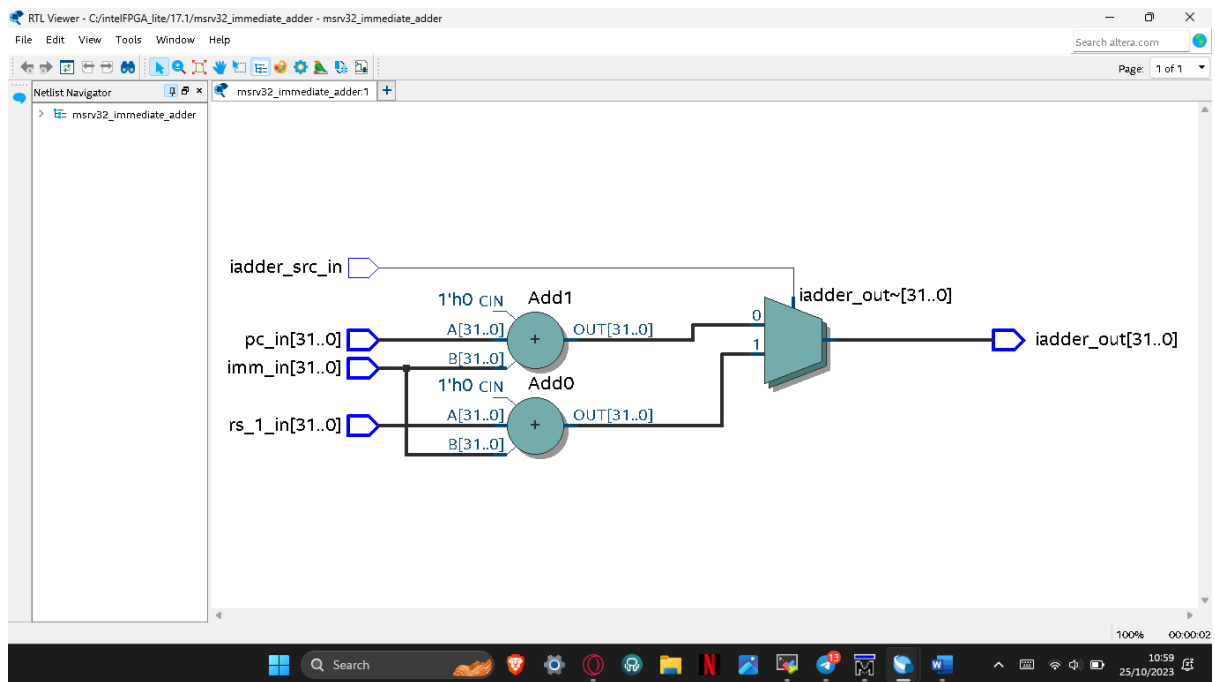
BLOCK DIAGRAM:



FUNCTIONALITY:

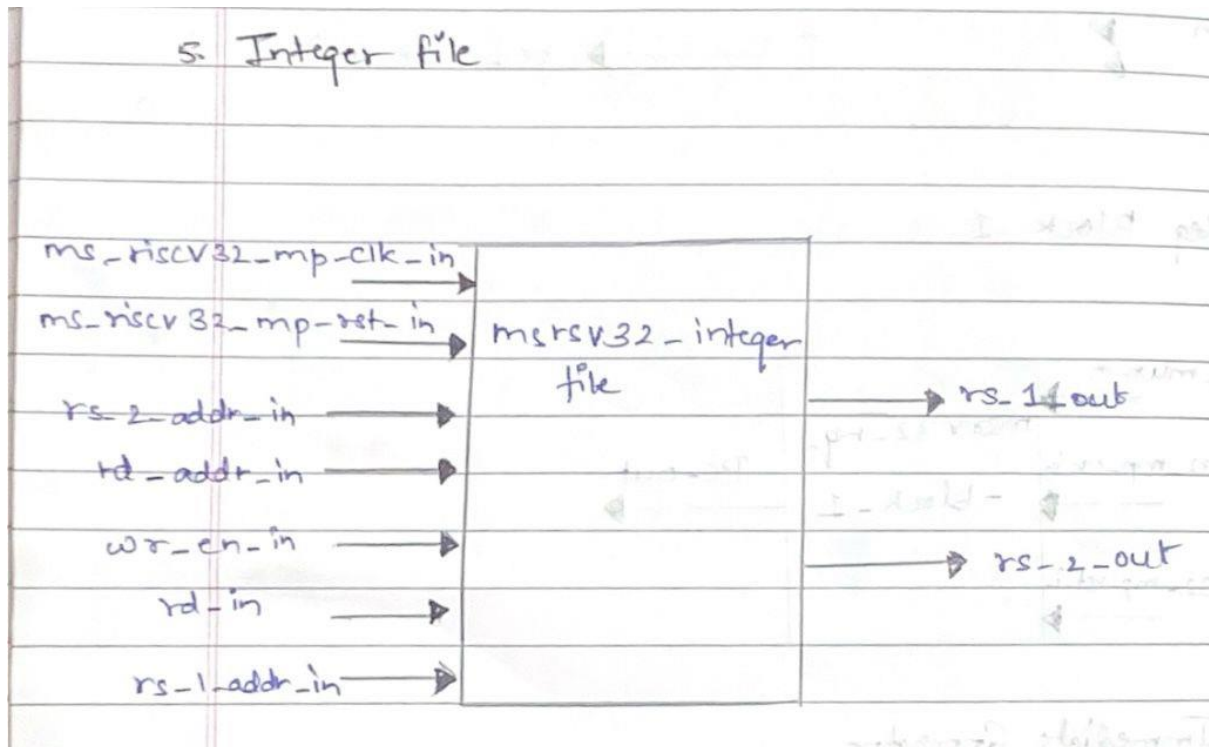
- It adds the immediate value to register value or pc value based on the requirement.
- This module adds immediate value to register value (rs1) when core wants to perform load, store, jump and link reg (jalr) instructions.
- The immediate adder produces an output that is the sum of the immediate value and the source register value
- immediate adder interprets the instruction type using the control signals. Based on the instruction type, it decides whether to add the immediate value to the register value (rs1) or the program counter (pc) value.

OUTPUT WAVEFORM:



5. INTEGER FILE

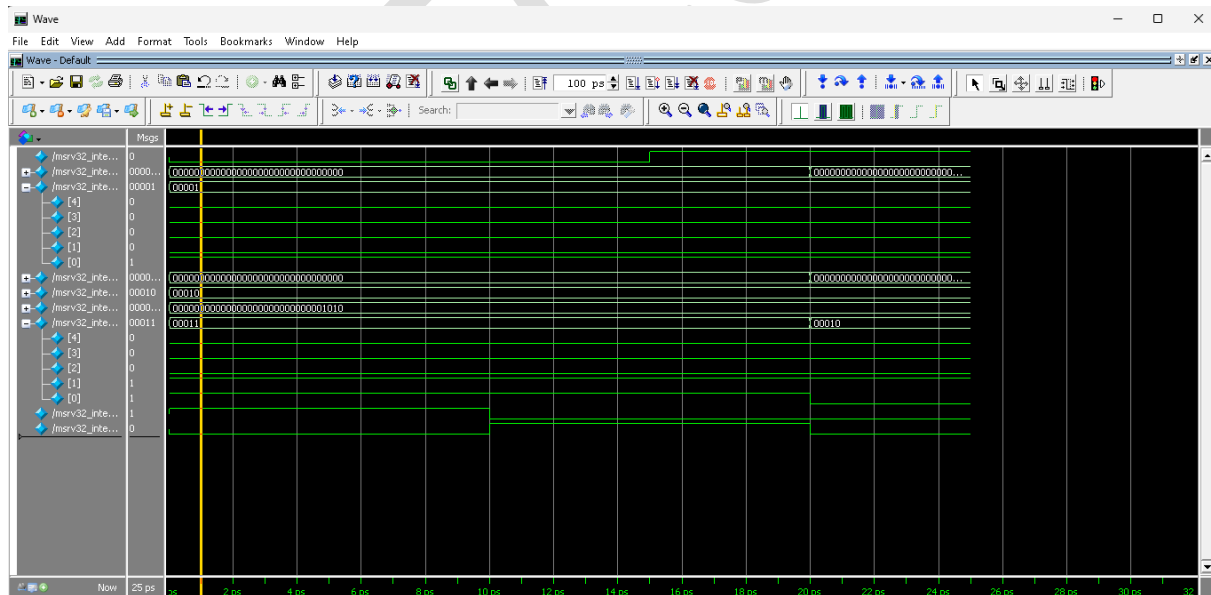
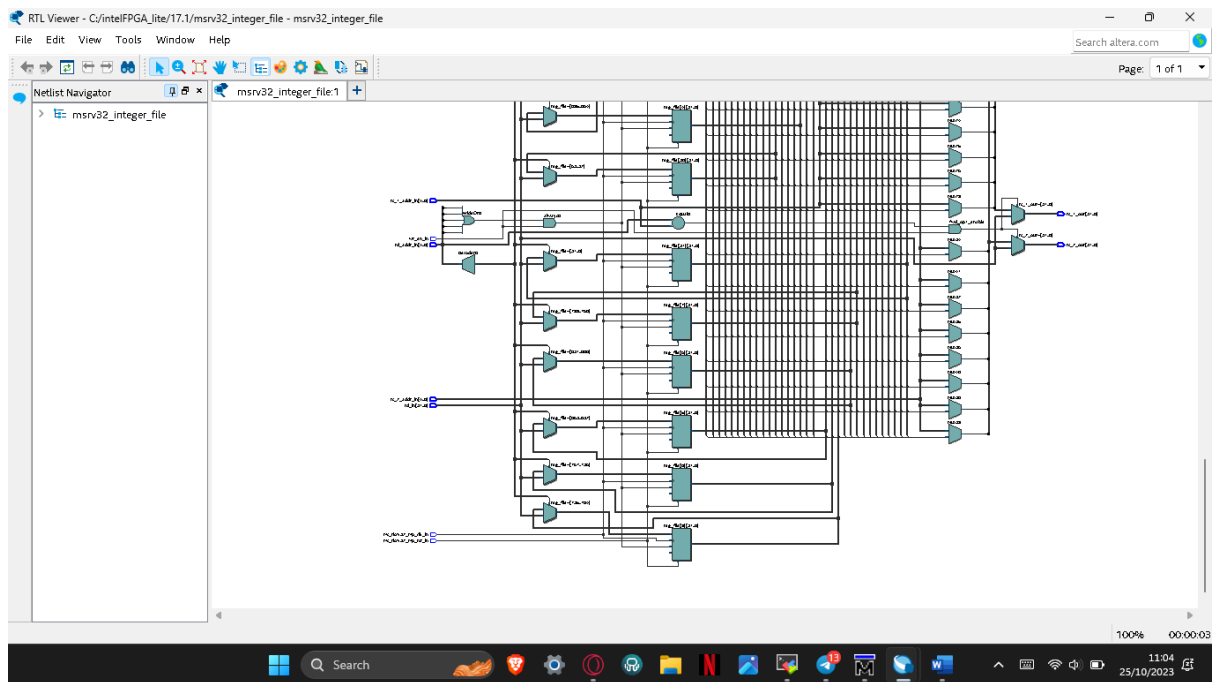
BLOCK DIAGRAM:



FUNCTIONALITY:

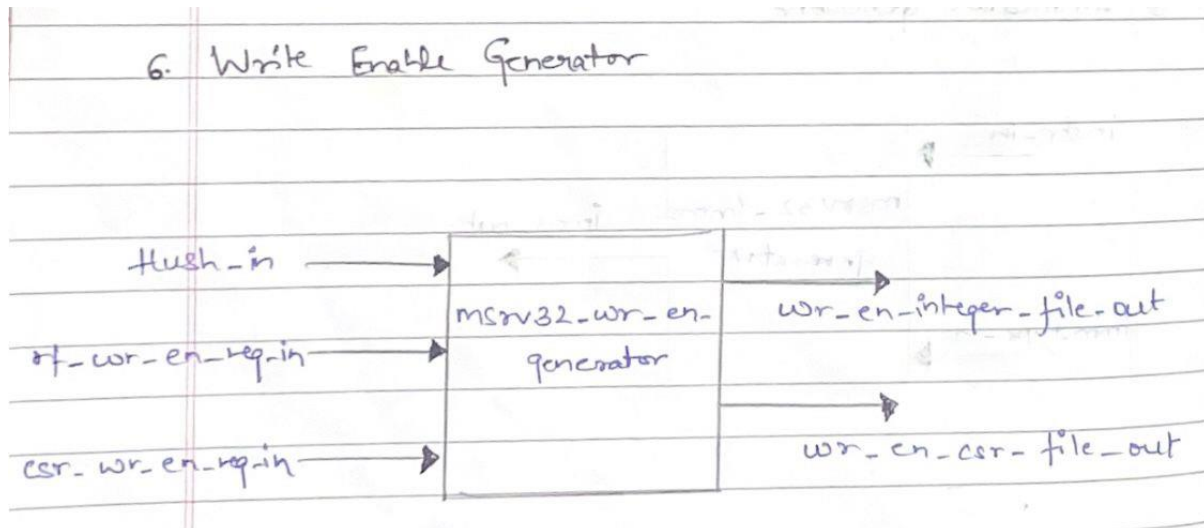
- This module supports read and write and provides access to general-purpose registers (GPRs) used for data storage and manipulation during program execution.
- It facilitates data read, data write, operand fetch, result writeback, register bypassing, and register reset operations.

OUTPUT WAVEFORM:



6. WRITE ENABLE GENERATOR

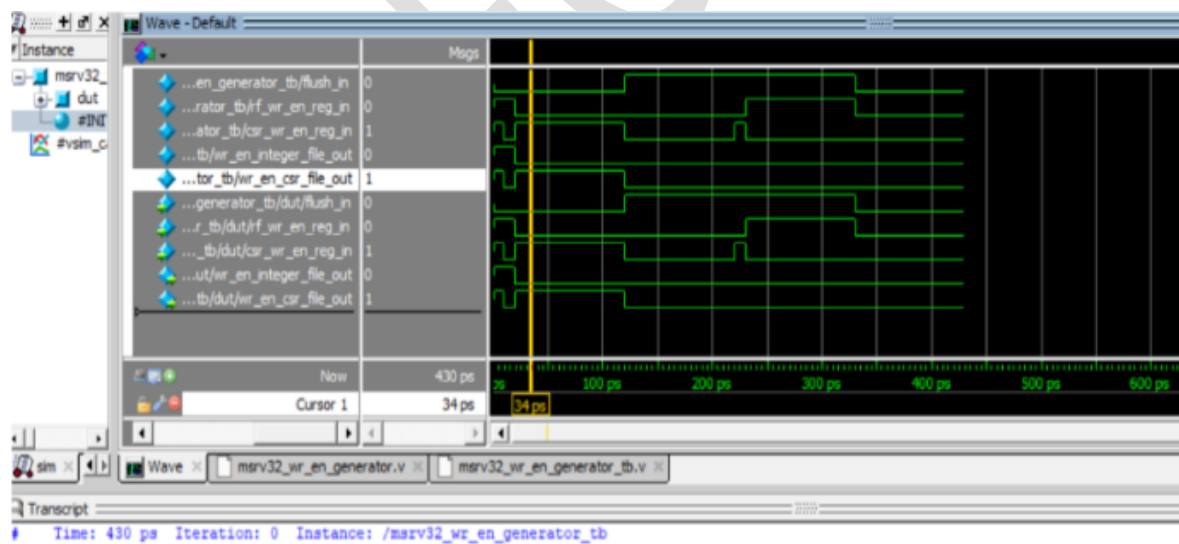
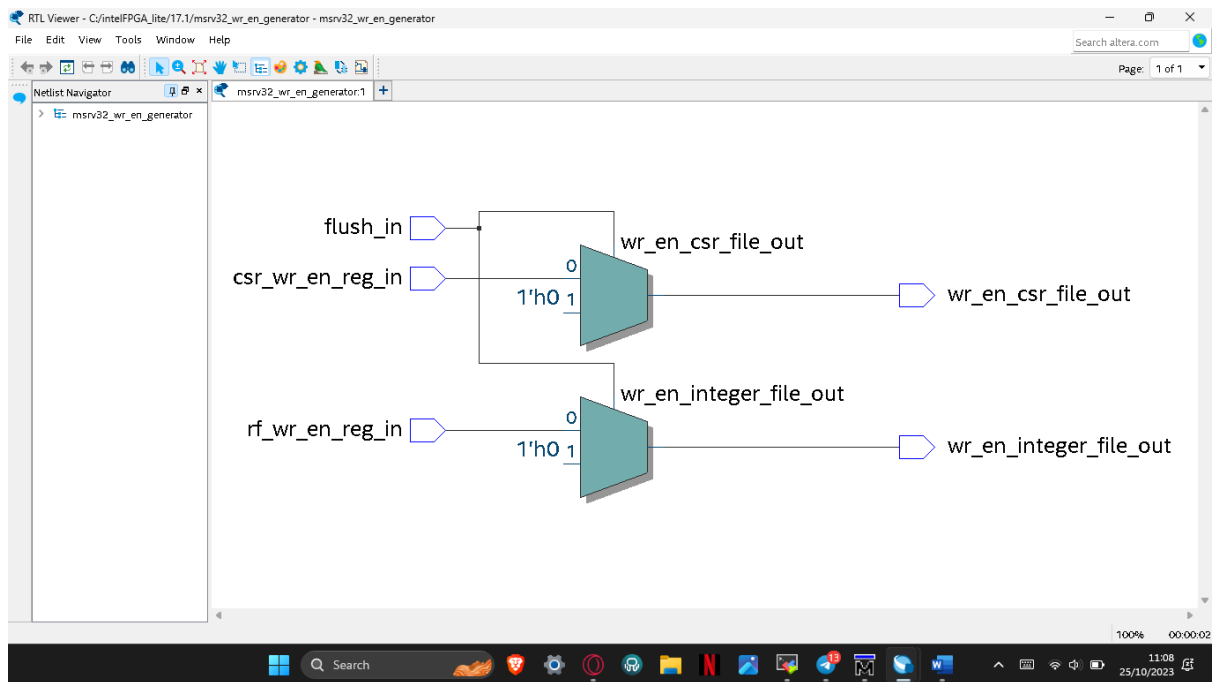
BLOCK DIAGRAM :



FUNCTIONALITY:

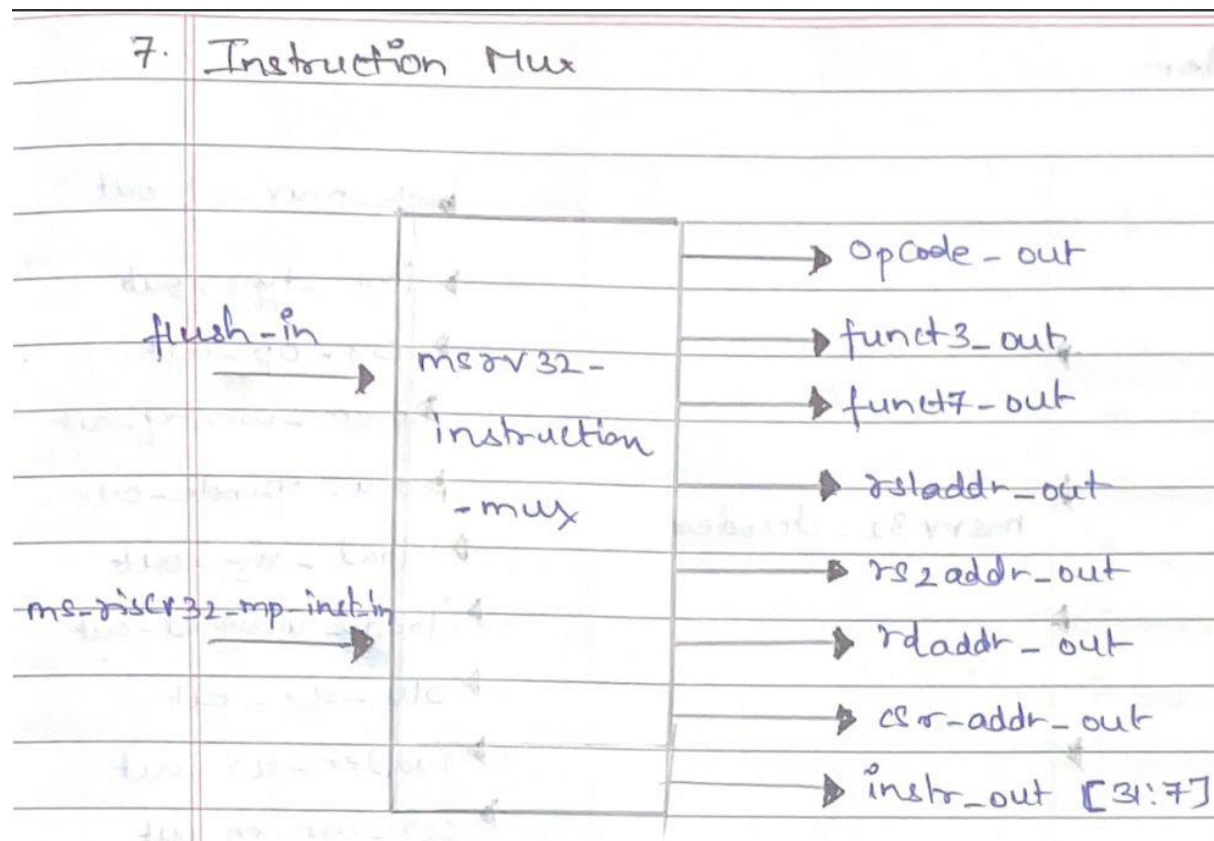
- The write enable generator in the RISC-V ISA determines whether a write operation should be enabled for a specific register based on the control signals and conditions provided.
- It generates the write enable signal that controls the writing of data into registers, ensuring that write operations occur only when necessary.
- It gives output based on flush in.

OUTPUT WAVEFORM:



7. INSTRUCTION MUX

BLOCK DIAGRAM :



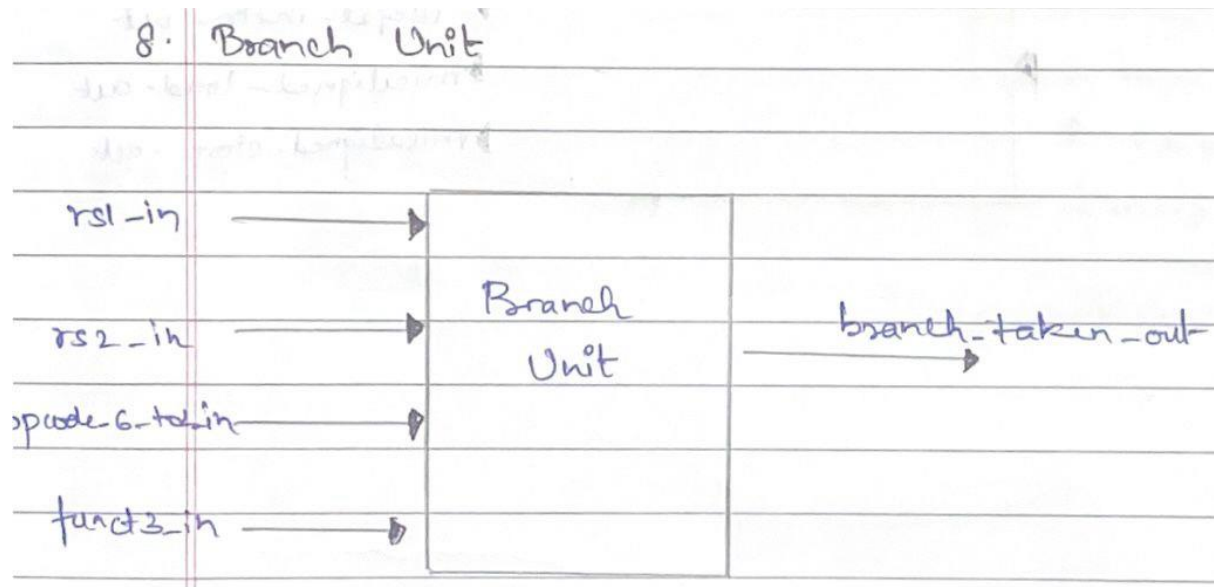
FUNCTIONALITY:

- The instruction mux in the RISC-V ISA selects the appropriate instruction to be executed based on control signals and the current execution stage.
- It routes the selected instruction to the relevant components for further processing, allowing for instruction decoding and execution.
- It takes instruction input and gives the output that serves as input to other modules to perform their functionality.



8. BRANCH UNIT

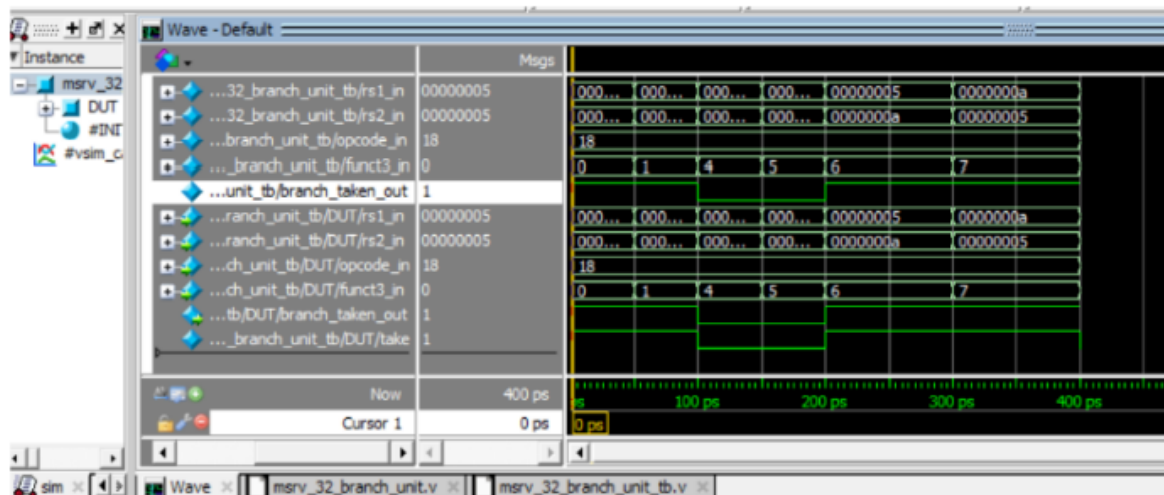
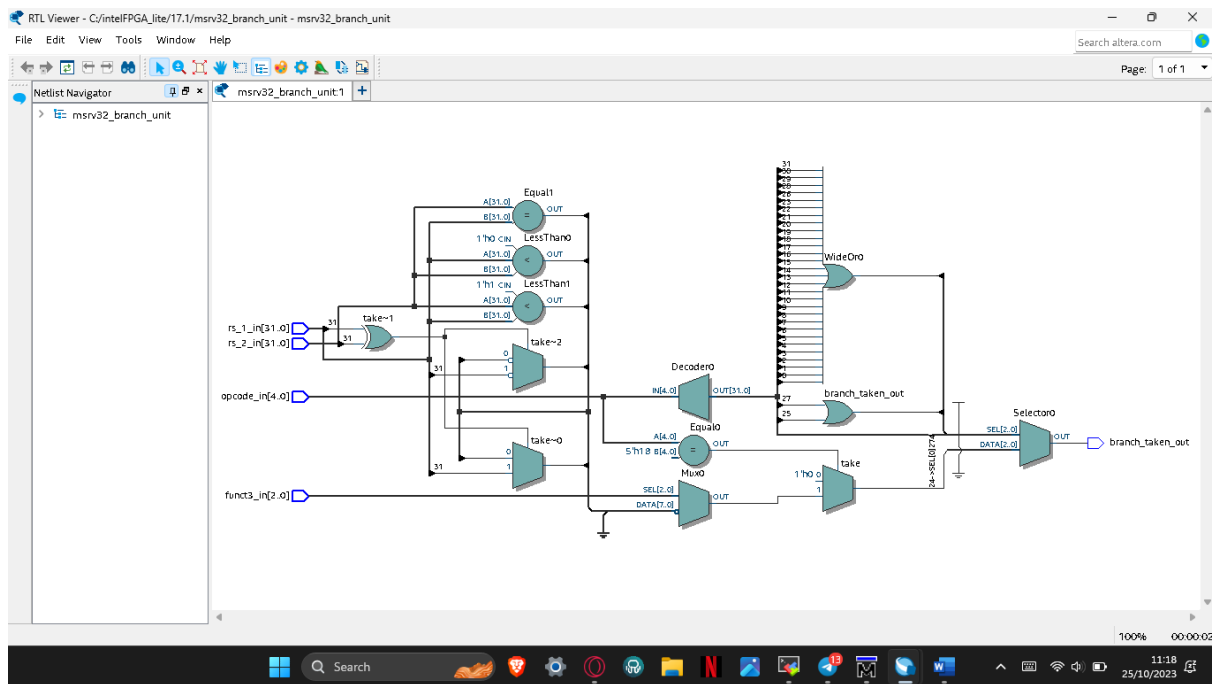
BLOCK DIAGRAM:



FUNCTIONALITY:

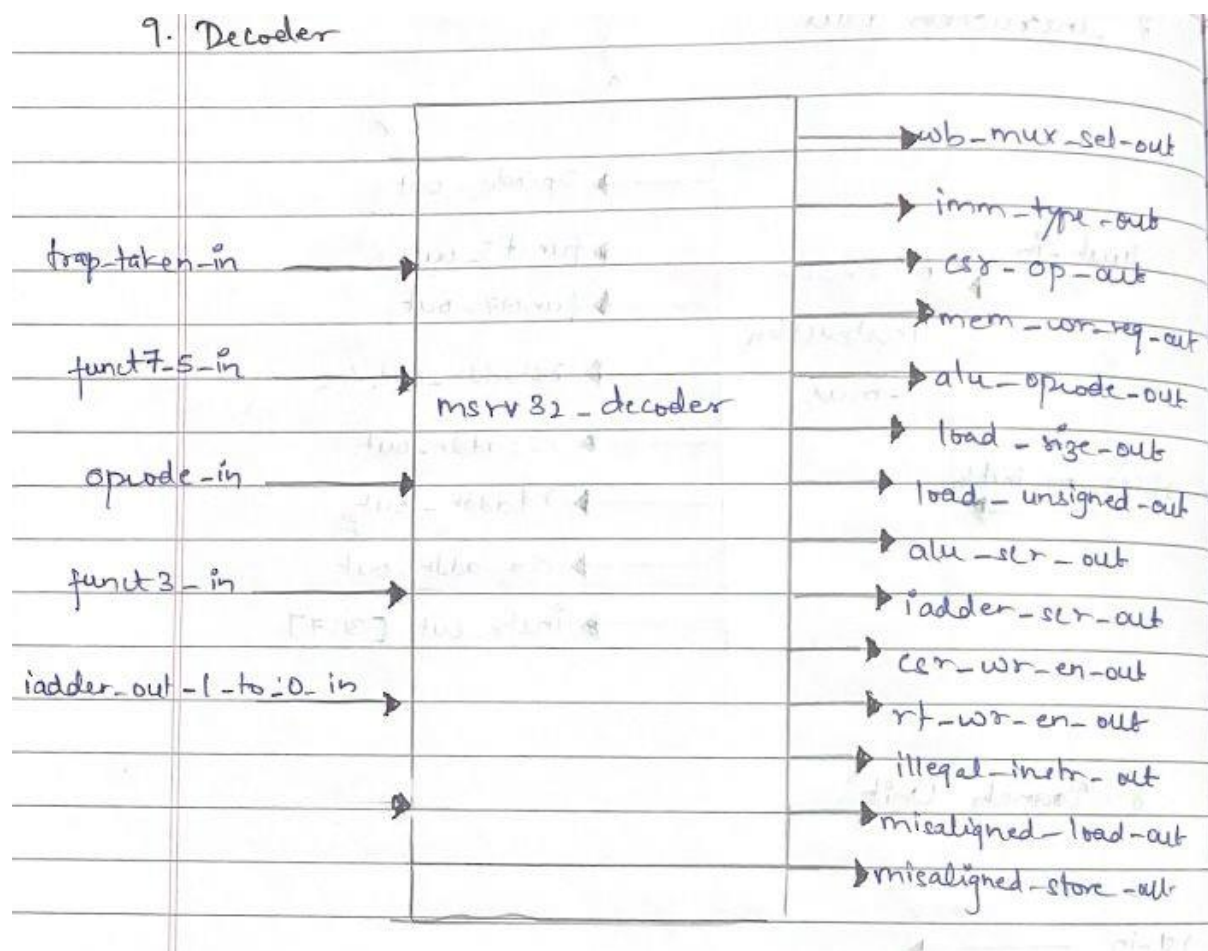
- It takes inputs from integer register file, based on opcode and function 3 values decides what kind of b type instruction that has to be executed.
- It evaluates branch conditions and determines whether a branch should be taken.
- It compares the values of registers or immediate values and generates control signals to update the program counter accordingly and enables conditional branching in the instruction flow.

OUTPUT WAVEFORM:



9. DECODER

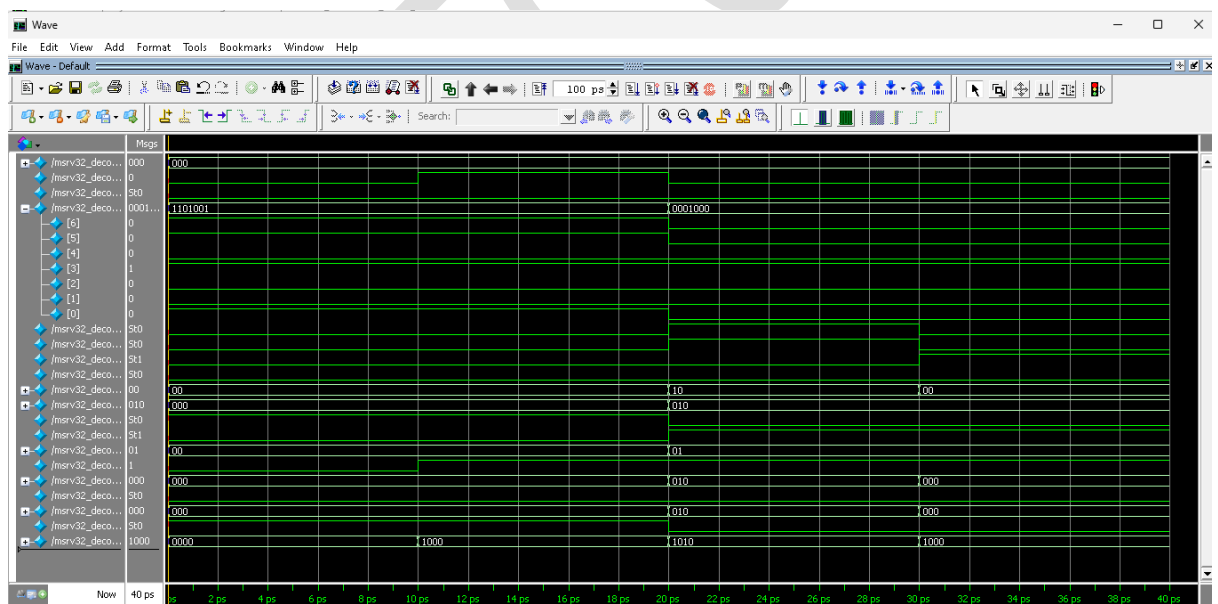
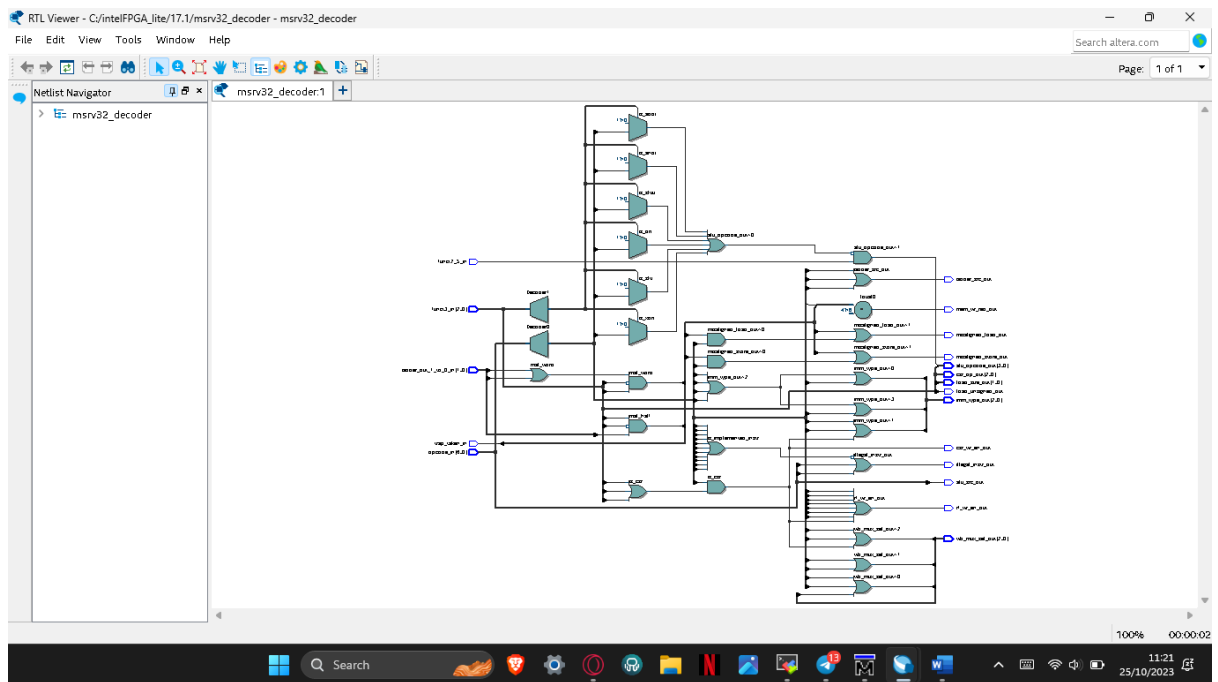
BLOCK DIAGRAM:



FUNCTIONALITY:

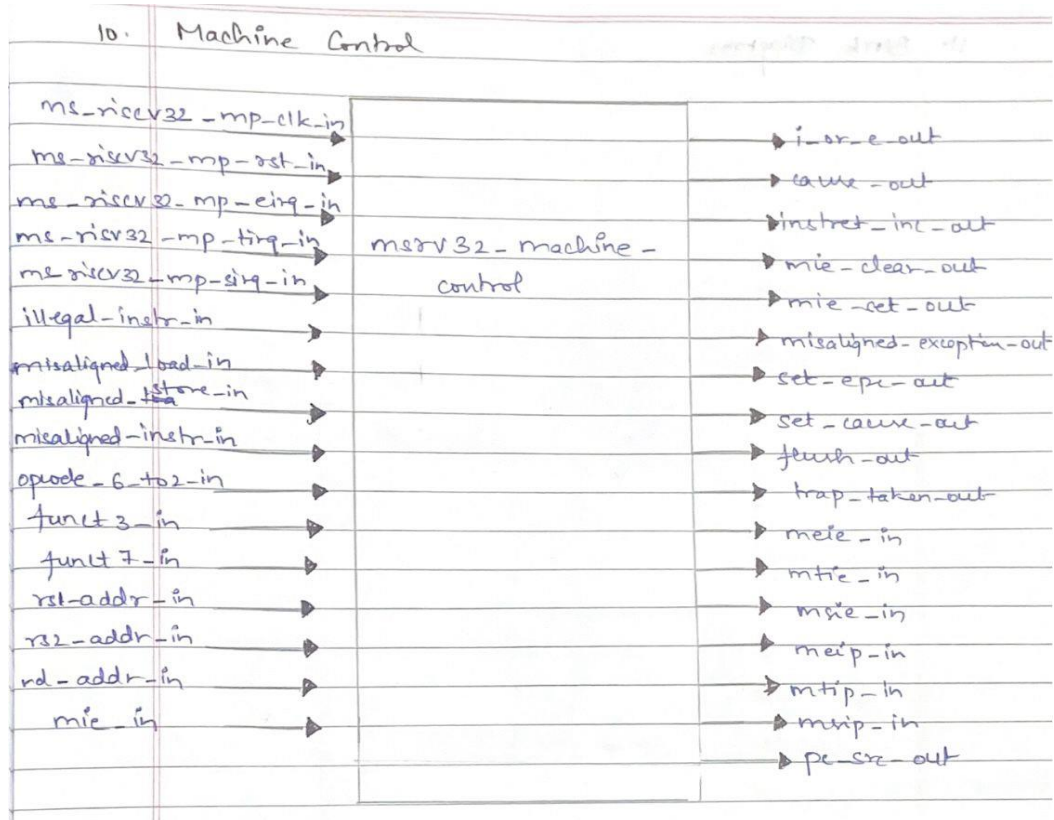
- Decoder is the main controller of RISC-V ISA design.
- It decodes the instruction fetched from memory and extracts the opcode and other relevant fields.
- It generates control signals based on the opcode to enable the execution of the corresponding instruction by coordinating various functional units within the processor.
- The generated signals control memory, load unit, store unit, ALU, two register files, immediate generator and write back multiplexer.

OUTPUT WAVEFORM:



10. MACHINE CONTROL

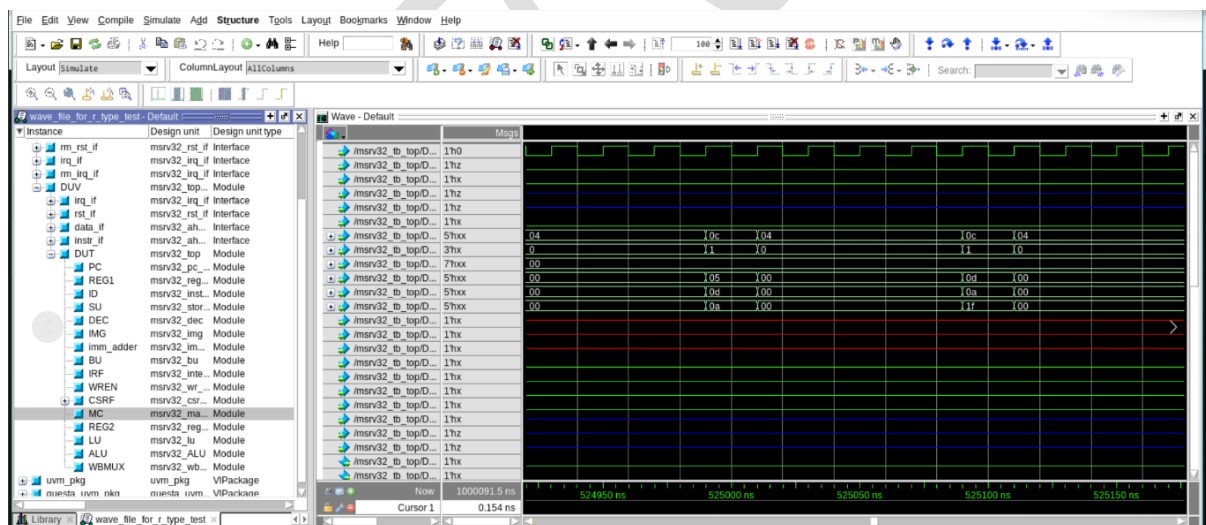
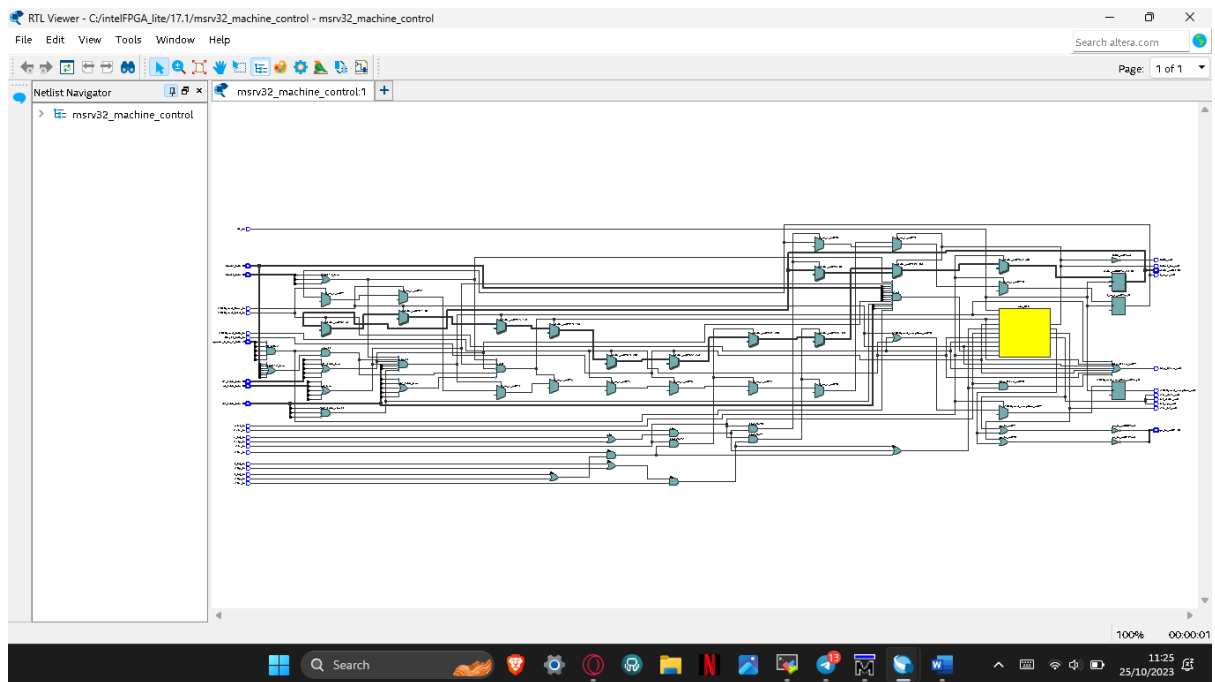
BLOCK DIAGRAM :



FUNCTIONALITY:

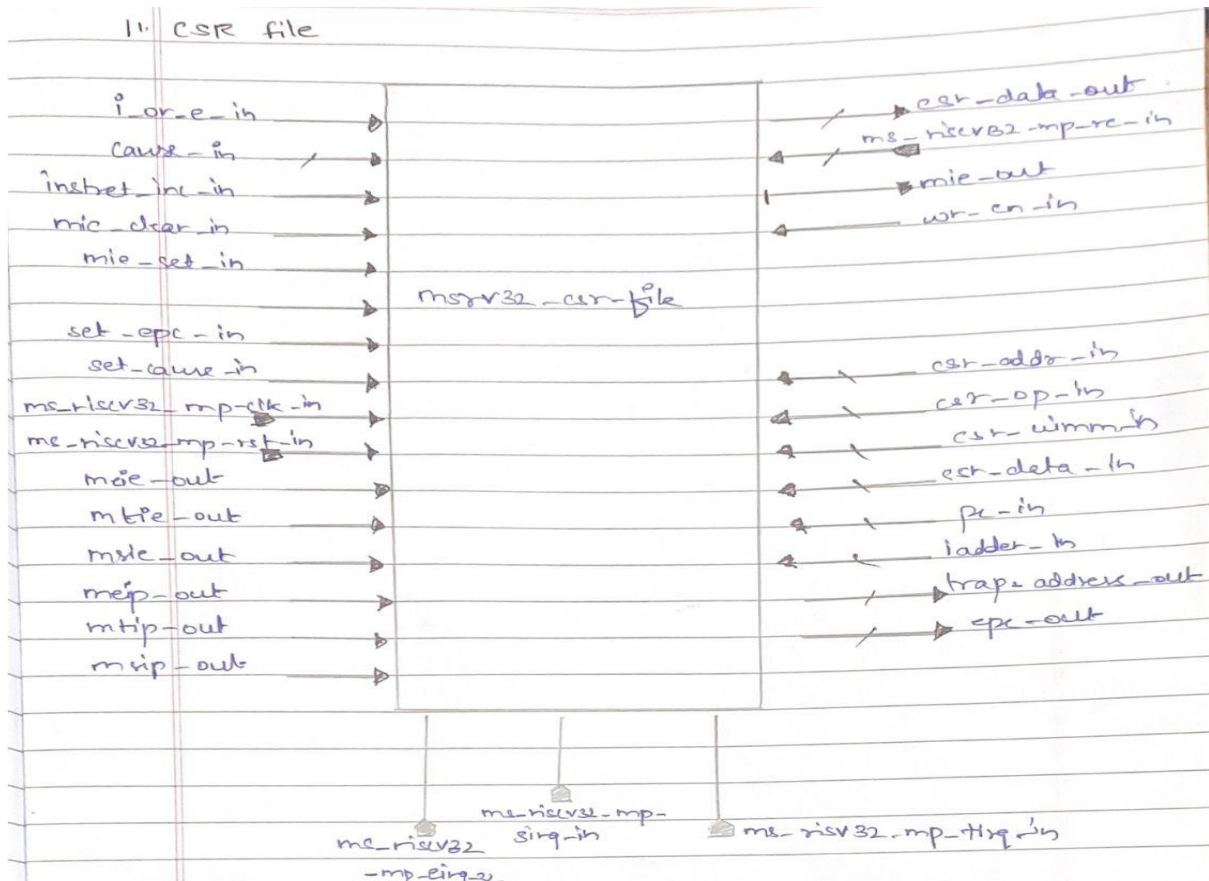
- Machine control decodes the instructions read from memory, extracting the opcode and other important data,
- generates control signals based on the opcode and extracted fields. These signals are utilized to coordinate instruction execution and to operate various functional units within the processor.
- Control signals generated by the processor are used to coordinate various functional components.
- machine control generates signals to regulate activities on two register files. During instruction execution, it coordinates the reading and writing of registers.

OUTPUT WAVEFORM:



11. CSR FILE

BLOCK DIAGRAM:

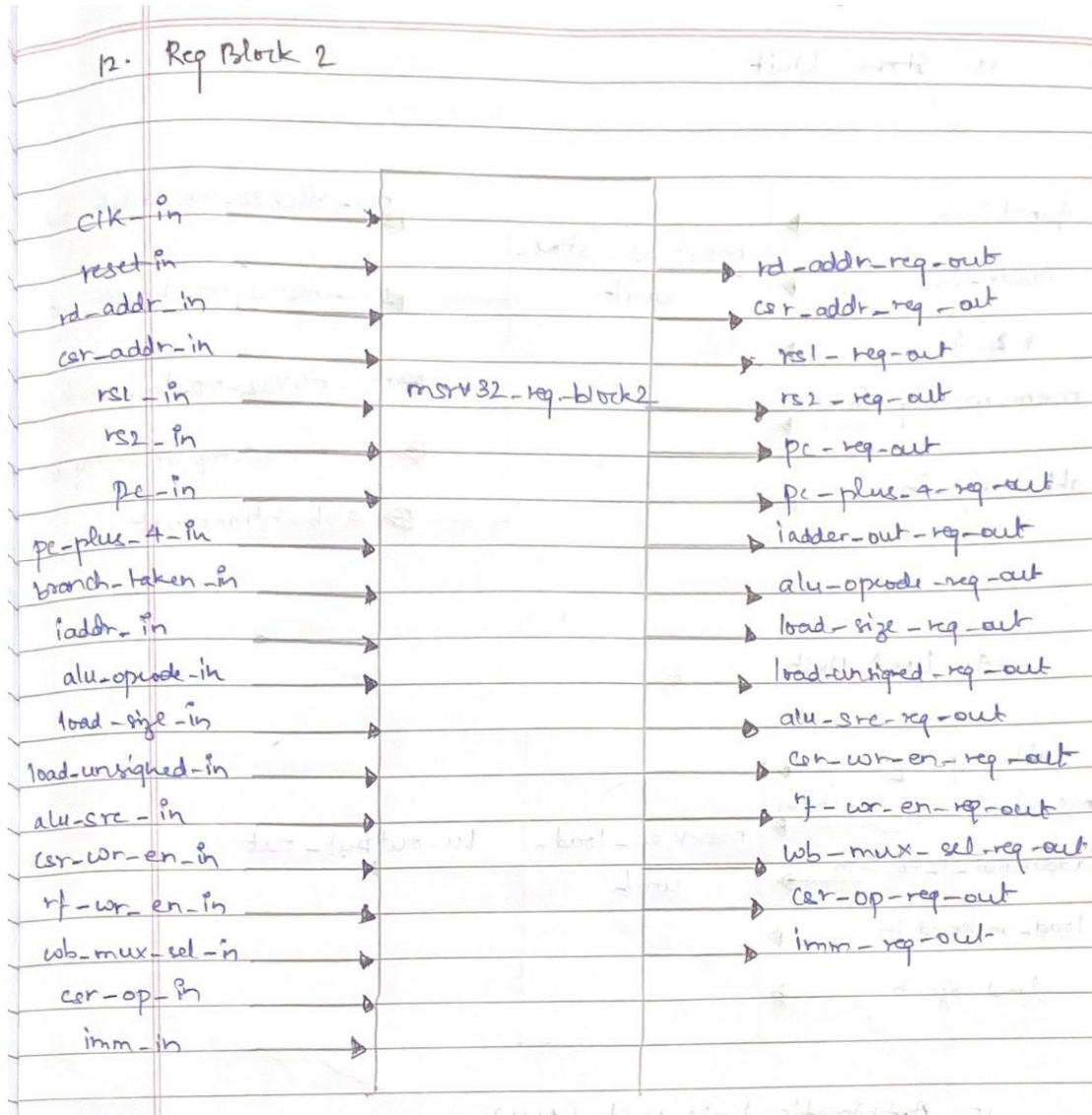


FUNCTIONALITY:

- The CSR file contains a collection of special-purpose registers that are used to control and monitor the processor's behavior.
- CSRs can be accessible at several levels of privilege, such as Machine mode (M-mode), Supervisor mode (S-mode), and User mode (U-mode). The privilege mode in which the processor is functioning determines access privileges.
- Reading a CSR returns the current value, but writing to a CSR changes the value.
- CSR-specific instructions enable the processor to read, write, and alter CSR values.

12. REG BLOCK 2

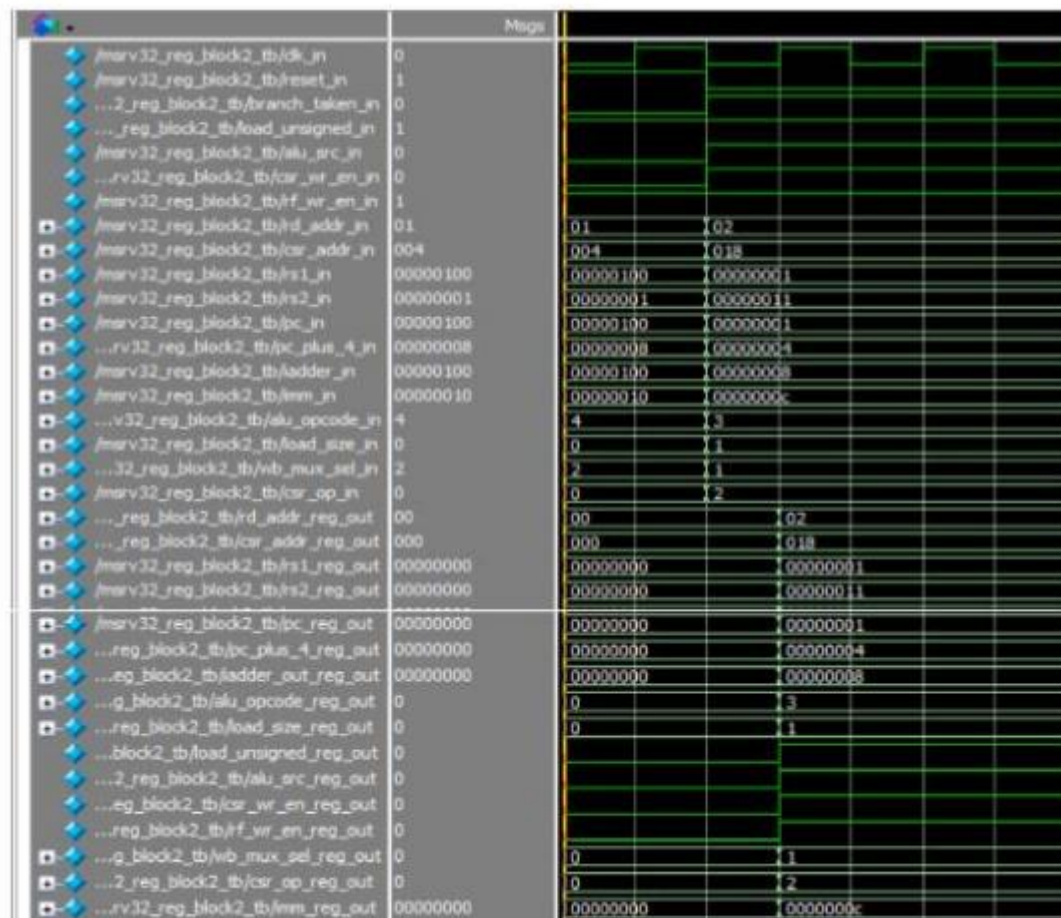
BLOCK DIAGRAM:



FUNCTIONALITY:

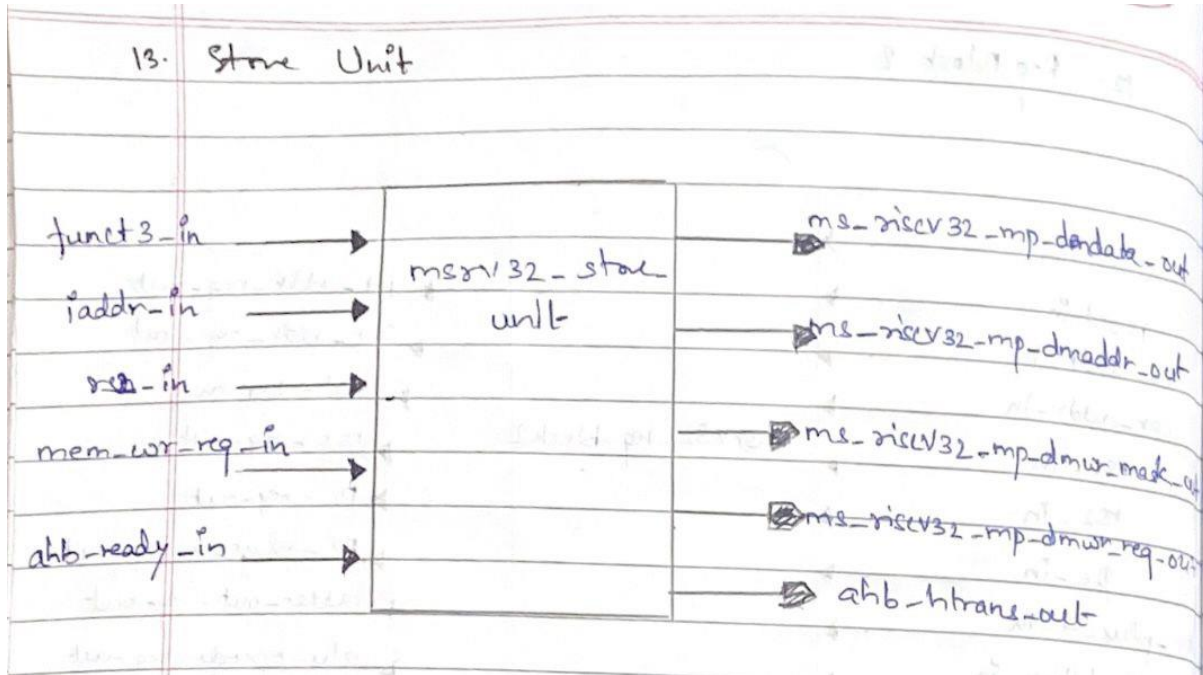
- It serves as a storage unit for storing temporary results and intermediate values during the execution of instructions.
- It provides read and write access to a set of registers, allowing instructions to retrieve data from and store data into specific registers as needed during program execution.

OUTPUT WAVEFORM:



13. STORE UNIT

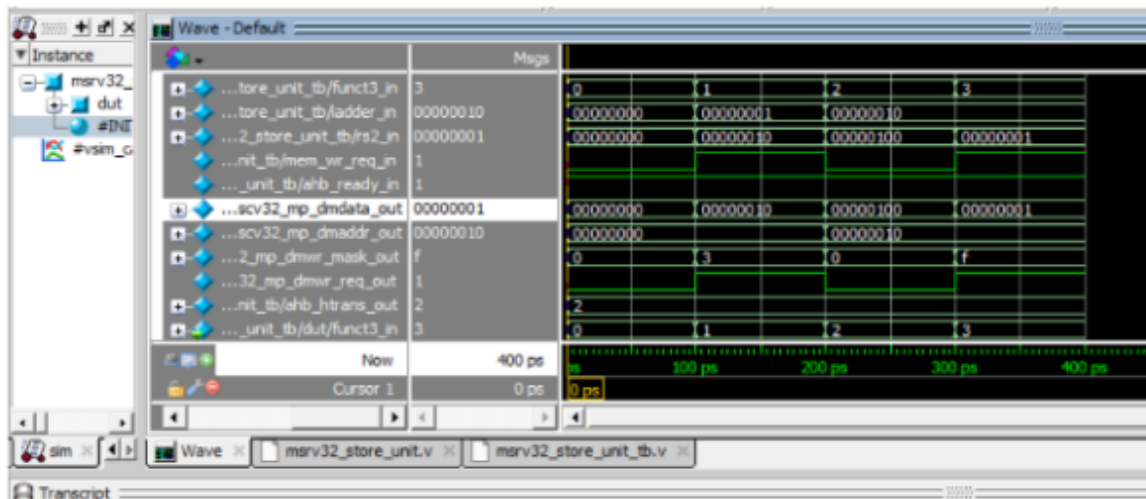
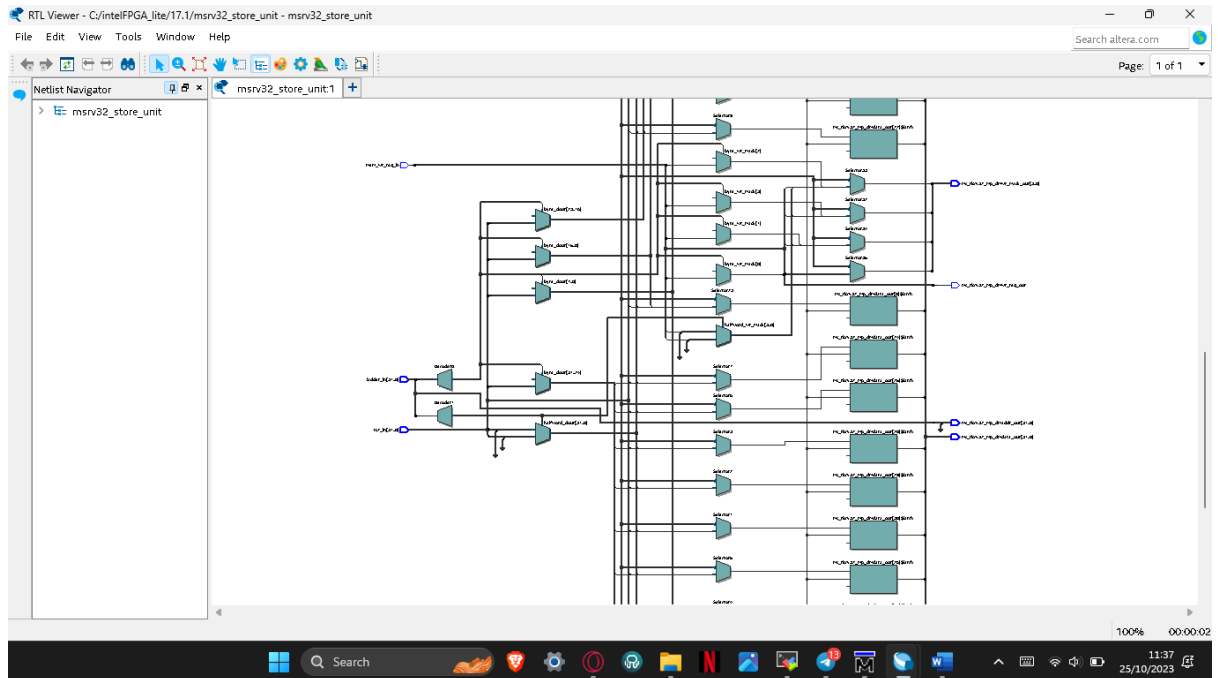
BLOCK DIAGRAM:



FUNCTIONALITY:

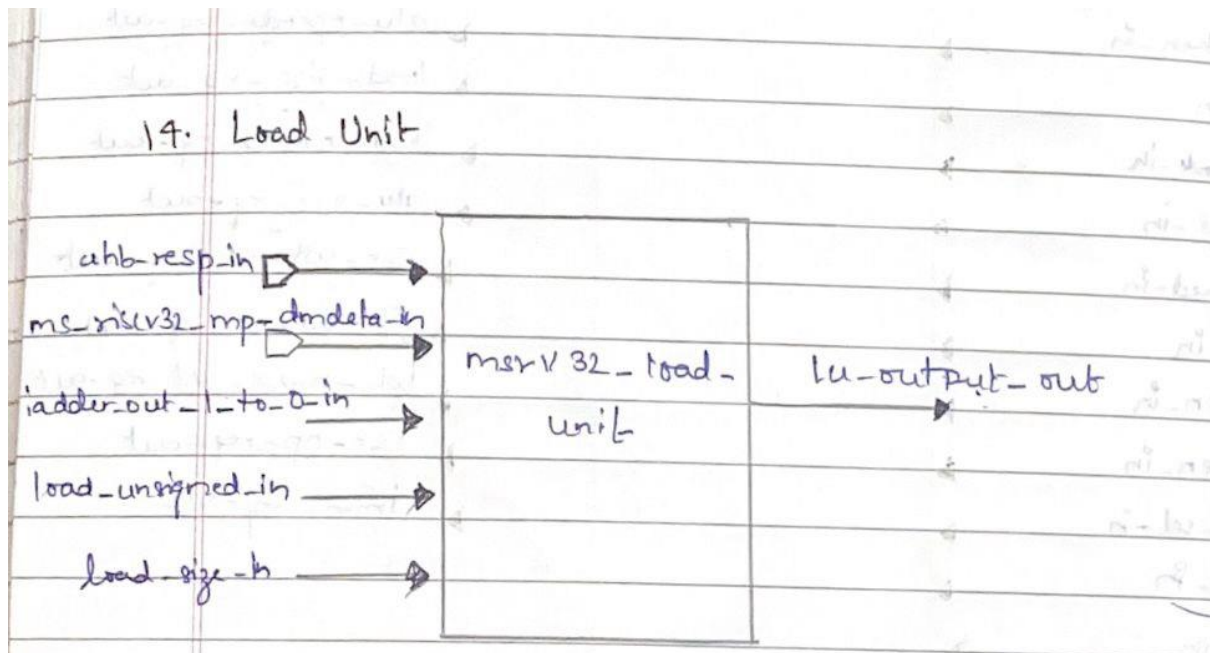
- It drives the signals that interface with external data memory.
- It executes store instructions.
- It transfers data from a source register to the specified memory location in the data memory.
- It performs the necessary address calculation and data transfer operations to store the data into memory

OUTPUT WAVEFORM:



14. LOAD UNIT

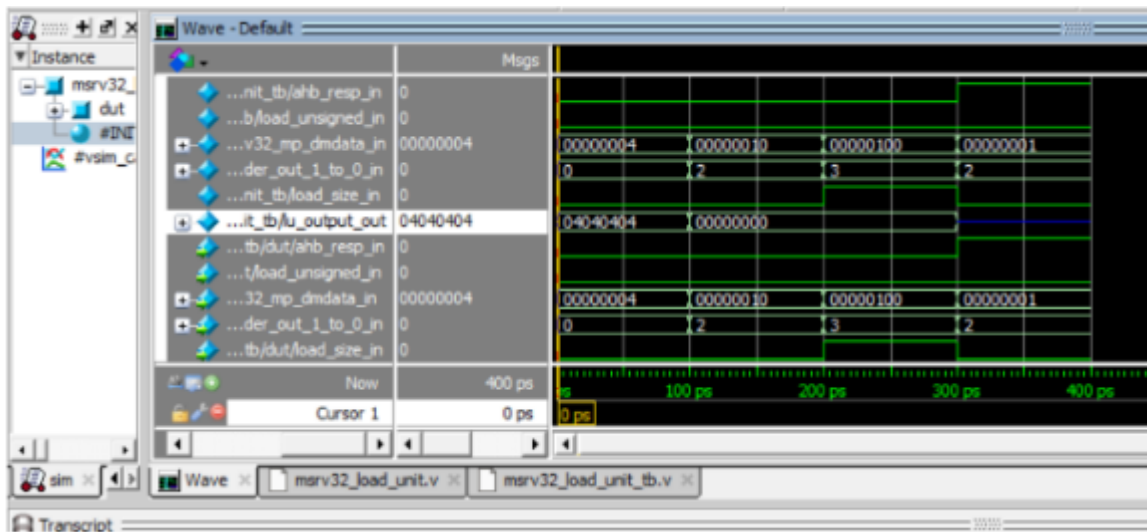
BLOCK DIAGRAM:



FUNCTIONALITY:

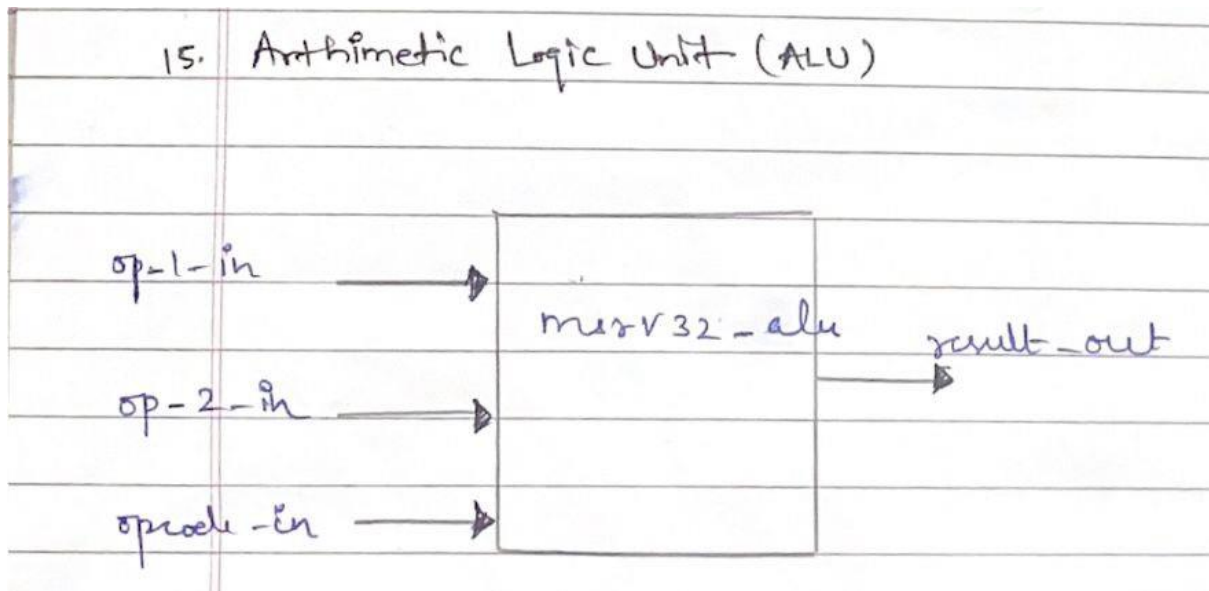
- Load unit is responsible for executing load instructions.
- It retrieves data from memory based on the specified memory location and transfers it to a destination register.
- It performs the necessary address calculation and data retrieval operations to load the data from memory

OUTPUT WAVEFORM:



15. ARITHMETIC LOGIC UNIT(ALU)

BLOCK DIAGRAM:



FUNCTIONALITY:

- The ALU performs various arithmetic and logical operations on operands provided by the register file.
- It supports operations such as addition, subtraction, bitwise operations, shifts, and comparisons.
- The ALU produces the result of the operation, which can be stored in a destination register or used for further computation.
- It checks the opcode and performs respective task accordingly.

RTL Viewer - C:/intelFPGA_lite/17.1/msrv32_alu - msnv32_alu

File Edit View Tools Window Help

Search altera.com

Page: 1 of 1

Netlist Navigator

msrv32_alu

msrv32_alu_1

msrv32_alu_2

op_1_in[31:0]

op_2_in[31:0]

#msrv32_alu_1

#msrv32_alu_2

#msrv32_alu_3

#msrv32_alu_4

#msrv32_alu_5

#msrv32_alu_6

#msrv32_alu_7

#msrv32_alu_8

#msrv32_alu_9

#msrv32_alu_10

#msrv32_alu_11

#msrv32_alu_12

#msrv32_alu_13

#msrv32_alu_14

#msrv32_alu_15

#msrv32_alu_16

#msrv32_alu_17

#msrv32_alu_18

#msrv32_alu_19

#msrv32_alu_20

#msrv32_alu_21

#msrv32_alu_22

#msrv32_alu_23

#msrv32_alu_24

#msrv32_alu_25

#msrv32_alu_26

#msrv32_alu_27

#msrv32_alu_28

#msrv32_alu_29

#msrv32_alu_30

#msrv32_alu_31

#msrv32_alu_32

#msrv32_alu_33

#msrv32_alu_34

#msrv32_alu_35

#msrv32_alu_36

#msrv32_alu_37

#msrv32_alu_38

#msrv32_alu_39

#msrv32_alu_40

#msrv32_alu_41

#msrv32_alu_42

#msrv32_alu_43

#msrv32_alu_44

#msrv32_alu_45

#msrv32_alu_46

#msrv32_alu_47

#msrv32_alu_48

#msrv32_alu_49

#msrv32_alu_50

#msrv32_alu_51

#msrv32_alu_52

#msrv32_alu_53

#msrv32_alu_54

#msrv32_alu_55

#msrv32_alu_56

#msrv32_alu_57

#msrv32_alu_58

#msrv32_alu_59

#msrv32_alu_60

#msrv32_alu_61

#msrv32_alu_62

#msrv32_alu_63

#msrv32_alu_64

#msrv32_alu_65

#msrv32_alu_66

#msrv32_alu_67

#msrv32_alu_68

#msrv32_alu_69

#msrv32_alu_70

#msrv32_alu_71

#msrv32_alu_72

#msrv32_alu_73

#msrv32_alu_74

#msrv32_alu_75

#msrv32_alu_76

#msrv32_alu_77

#msrv32_alu_78

#msrv32_alu_79

#msrv32_alu_80

#msrv32_alu_81

#msrv32_alu_82

#msrv32_alu_83

#msrv32_alu_84

#msrv32_alu_85

#msrv32_alu_86

#msrv32_alu_87

#msrv32_alu_88

#msrv32_alu_89

#msrv32_alu_90

#msrv32_alu_91

#msrv32_alu_92

#msrv32_alu_93

#msrv32_alu_94

#msrv32_alu_95

#msrv32_alu_96

#msrv32_alu_97

#msrv32_alu_98

#msrv32_alu_99

#msrv32_alu_100

#msrv32_alu_101

#msrv32_alu_102

#msrv32_alu_103

#msrv32_alu_104

#msrv32_alu_105

#msrv32_alu_106

#msrv32_alu_107

#msrv32_alu_108

#msrv32_alu_109

#msrv32_alu_110

#msrv32_alu_111

#msrv32_alu_112

#msrv32_alu_113

#msrv32_alu_114

#msrv32_alu_115

#msrv32_alu_116

#msrv32_alu_117

#msrv32_alu_118

#msrv32_alu_119

#msrv32_alu_120

#msrv32_alu_121

#msrv32_alu_122

#msrv32_alu_123

#msrv32_alu_124

#msrv32_alu_125

#msrv32_alu_126

#msrv32_alu_127

#msrv32_alu_128

#msrv32_alu_129

#msrv32_alu_130

#msrv32_alu_131

#msrv32_alu_132

#msrv32_alu_133

#msrv32_alu_134

#msrv32_alu_135

#msrv32_alu_136

#msrv32_alu_137

#msrv32_alu_138

#msrv32_alu_139

#msrv32_alu_140

#msrv32_alu_141

#msrv32_alu_142

#msrv32_alu_143

#msrv32_alu_144

#msrv32_alu_145

#msrv32_alu_146

#msrv32_alu_147

#msrv32_alu_148

#msrv32_alu_149

#msrv32_alu_150

#msrv32_alu_151

#msrv32_alu_152

#msrv32_alu_153

#msrv32_alu_154

#msrv32_alu_155

#msrv32_alu_156

#msrv32_alu_157

#msrv32_alu_158

#msrv32_alu_159

#msrv32_alu_160

#msrv32_alu_161

#msrv32_alu_162

#msrv32_alu_163

#msrv32_alu_164

#msrv32_alu_165

#msrv32_alu_166

#msrv32_alu_167

#msrv32_alu_168

#msrv32_alu_169

#msrv32_alu_170

#msrv32_alu_171

#msrv32_alu_172

#msrv32_alu_173

#msrv32_alu_174

#msrv32_alu_175

#msrv32_alu_176

#msrv32_alu_177

#msrv32_alu_178

#msrv32_alu_179

#msrv32_alu_180

#msrv32_alu_181

#msrv32_alu_182

#msrv32_alu_183

#msrv32_alu_184

#msrv32_alu_185

#msrv32_alu_186

#msrv32_alu_187

#msrv32_alu_188

#msrv32_alu_189

#msrv32_alu_190

#msrv32_alu_191

#msrv32_alu_192

#msrv32_alu_193

#msrv32_alu_194

#msrv32_alu_195

#msrv32_alu_196

#msrv32_alu_197

#msrv32_alu_198

#msrv32_alu_199

#msrv32_alu_200

#msrv32_alu_201

#msrv32_alu_202

#msrv32_alu_203

#msrv32_alu_204

#msrv32_alu_205

#msrv32_alu_206

#msrv32_alu_207

#msrv32_alu_208

#msrv32_alu_209

#msrv32_alu_210

#msrv32_alu_211

#msrv32_alu_212

#msrv32_alu_213

#msrv32_alu_214

#msrv32_alu_215

#msrv32_alu_216

#msrv32_alu_217

#msrv32_alu_218

#msrv32_alu_219

#msrv32_alu_220

#msrv32_alu_221

#msrv32_alu_222

#msrv32_alu_223

#msrv32_alu_224

#msrv32_alu_225

#msrv32_alu_226

#msrv32_alu_227

#msrv32_alu_228

#msrv32_alu_229

#msrv32_alu_230

#msrv32_alu_231

#msrv32_alu_232

#msrv32_alu_233

#msrv32_alu_234

#msrv32_alu_235

#msrv32_alu_236

#msrv32_alu_237

#msrv32_alu_238

#msrv32_alu_239

#msrv32_alu_240

#msrv32_alu_241

#msrv32_alu_242

#msrv32_alu_243

#msrv32_alu_244

#msrv32_alu_245

#msrv32_alu_246

#msrv32_alu_247

#msrv32_alu_248

#msrv32_alu_249

#msrv32_alu_250

#msrv32_alu_251

#msrv32_alu_252

#msrv32_alu_253

#msrv32_alu_254

#msrv32_alu_255

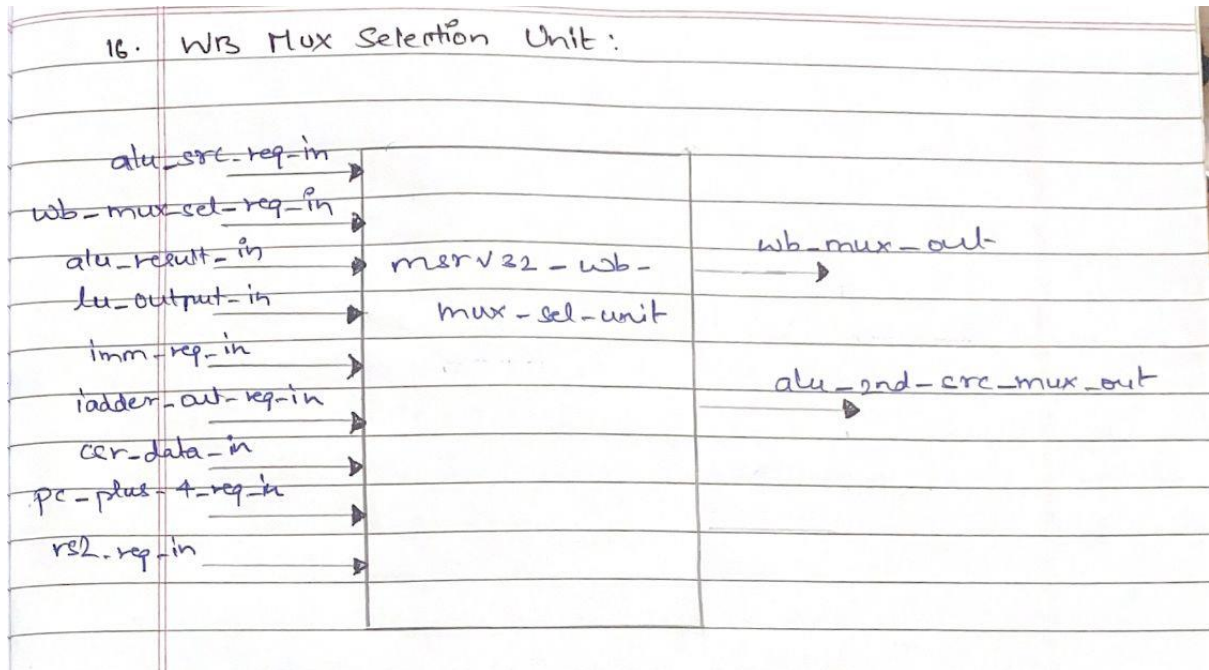
#msrv32_alu_256

#msrv32_al



16. WB MUX SELECTION UNIT

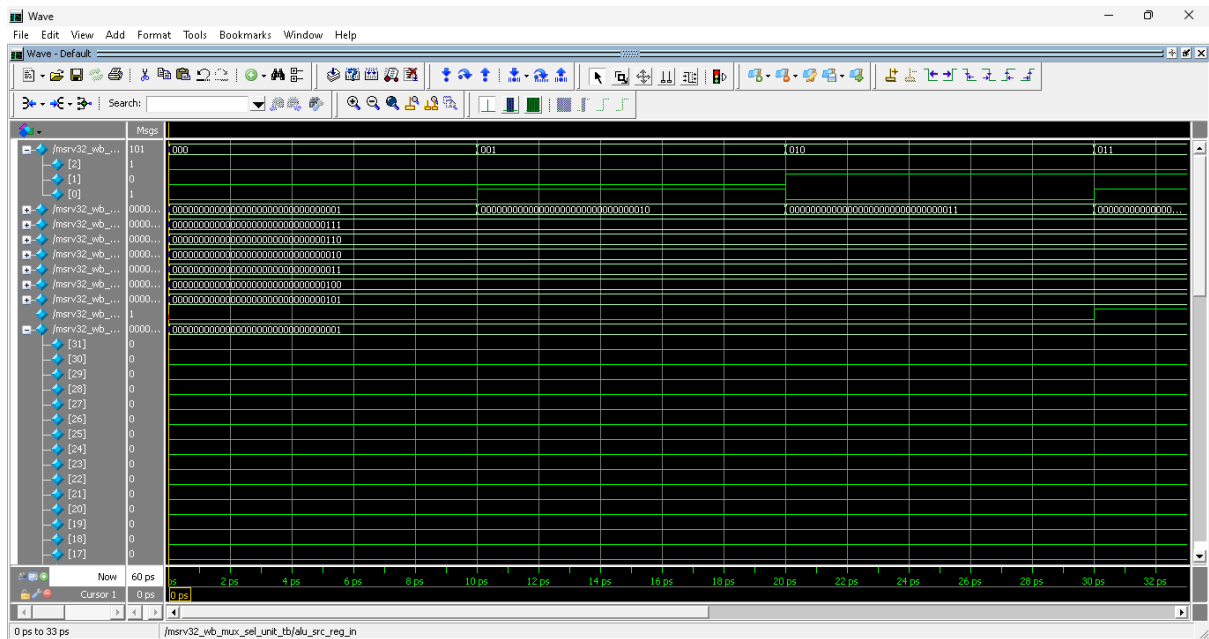
BLOCK DIAGRAM:



FUNCTIONALITY:

- It selects the data to be written back from different sources based on control signals.
- It determines whether the data from the ALU output or the memory unit (load unit) is forwarded to the destination register for storage i.e., write-back mux unit selects whether to write ALU output, load unit back to instruction memory.
- The selection is typically based on the instruction type and control signals to ensure the correct data is written back to the register file.

OUTPUT WAVEFORM:



CONCLUSION :

In conclusion, the completion of the project focused on the **RISC-V RV321 architecture** is done.

I successfully designed and analyzed the RISC-V RV321 architecture, gaining profound insights into the complexities of modern microprocessor systems.

This initiative opened doors for future research and developments in the fascinating topic of computer architecture by providing an entry point into the field of processor design.

The Project deepened my understanding of microprocessor architectures and also honed my skills in utilizing software tools such as **Model Sim Altera, Quartus Prime, Sophus Connect, and MobaXTerm.**